

Diplomarbeit

Smart-Mirror

Ein Spiegel auf Industriestandard 4.0

erstellt von

Mario Petrovic



HTBLuVA
Innsbruck Anichstrasse

Betreuer:
Albert Greinöcker, Mag. Dr. DI

2020/21

Eidesstattliche Erklärung

Ich erkläre an Eides statt, dass ich die vorliegende Diplomarbeit selbstständig und ohne fremde Hilfe verfasst, andere als die angegebenen Quellen und Hilfsmittel nicht benutzt und die den benutzten Quellen wörtlich und inhaltlich entnommenen Stellen als solche kenntlich gemacht habe.

Verfasser/Verfasserinnen:

Innsbruck, am 18. März 2021

Mario Petrovic

Mario Petrovic

Einverständniserklärung

Ich bin damit einverstanden, dass ein Exemplar dieser Arbeit mit dem Titel

*Smart-Mirror:
Ein Spiegel auf Industriestandard 4.0*

in der Bibliothek der Höheren Technischen Bundeslehr- und Versuchsanstalt Innsbruck Anichstraße ausgestellt wird.

Innsbruck, am 18. März 2021

Mario Petrovic

Mario Petrovic

Projektteam

Schüler



Mario Petrovic
Ziegeleiweg 7a
A-6361 Hopfgarten i.Bt.

Tel: +43 664 5482904
E-Mail: mariopetrovic02@hotmail.com

Betreuer



Mag. Dr. DI Albert Greinöcker
E-Mail: albert.greinoecker@htlinn.ac.at

Gender Erklärung

Aus Gründen der besseren Lesbarkeit wird in dieser Diplomarbeit durchweg die Sprachform des generischen Maskulinums angewendet. Es wird an dieser Stelle darauf hingewiesen, dass die ausschließliche Verwendung der männlichen Sprachform geschlechtsunabhängig verstanden werden soll.

Vorwort

Diese Diplomarbeit wurde im Schuljahr 2020/2021 im Rahmen der Reife- und Diplomprüfung an der Höheren Technischen Bundes-, Lehr- und Versuchsanstalt Innsbruck, Abteilung Wirtschaftsingenieurwesen mit Ausbildungsschwerpunkt Betriebsinformatik durchgeführt.

Bei unserer Diplomarbeit war uns vor allem wichtig, dass unser Projekt zwei grundlegende Säulen unserer Ausbildung abdeckt und kombiniert - Informatik und Maschinenbau. Wir wollten ein Hardware-Produkt herstellen, welches im Herzen ein softwaretechnisches Gerüst besitzt.

Bereits im vierten Jahrgang unserer Ausbildung suchten wir eifrig ein passendes Thema für unsere Diplomarbeit, welches uns gleichermaßen sowohl interessiert als auch fordert.

Die Grundidee kam schlussendlich von Mario Petrovic, als er eines Tages auf der Heimfahrt von der Schule im Internet surfte und ihm ein Video von einem Spiegel mit Touchscreen gesteuertem Display empfohlen wurde. Nach Absprache mit den anderen Teammitgliedern stand fest, dass wir unsere eigene Version von einem solchen Spiegel erstellen wollten, da dieses Projekt sowohl softwaretechnische als auch hardwaretechnische Elemente beinhaltet.

Da dieses Projekt vor allem sehr softwaretechnisch ist war uns klar, dass wir einen Betreuungslehrer benötigen, welcher in diesem Bereich schon viel Praxiserfahrung und Know-How besitzt. Daher fragten wir Herrn Mag. Dr. DI Albert Greinöcker ob er diese Rolle übernimmt. Dieser willigte auf unser Anliegen sofort ein.

Kurzbeschreibung

Im Zuge unserer Diplomarbeit entwickelten wir eine Software, welche die aktuellen Wetterdaten für einen gewünschten Standort ermittelt und abbildet. Die Wahl des Wunschstandorts wird durch eine Datenbank benutzerabhängig geregelt. Diese Software läuft auf einem Microcomputer, welcher auch über Internetzugang verfügt. Somit ist es dem Nutzer möglich im Internet zu surfen. All dies wird in ein Gerüst mit eingebautem Einwegspiegel integriert. Hinter diesem Einwegspiegel befindet sich ein Monitor, welcher die Anzeige unseres Microcomputers wiedergibt. Die Steuerung des Computers erfolgt über Gesichtserkennung und Gestiksteuerung.

Unsere Diplomarbeit umfasst somit einen **softwaretechnischen** sowie einen **praktischen** Teil:

Der **softwaretechnische** Teil umfasst die Entwicklung einer App, welche aktuelle Wetterdaten aus einer API ermittelt und darstellt. Zudem sollen die Wetterdaten anhand von Accounts ermittelt werden und Nutzer sollen im Internet surfen können. Zur Navigation dient eine Software, welche die Befehle des Nutzers mittels Gesicht- und Gestiksteuerung wahrnimmt.

Der Programmcode dieser Diplomarbeit wurde auf GitHub veröffentlicht und kann über den Link <https://github.com/htl-anichstrasse/smart-mirror> heruntergeladen werden.

Der **praktische** Teil umfasst alle hardwarerelevanten Kriterien in diesem Projekt. Hierunter fällt einerseits die Beschaffung der benötigten Hardware und andererseits die Erstellung einer Konstruktionszeichnung für den Spiegelrahmen und dessen Fertigung.

Abstract

In the course of our diploma project we developed a software, which determines and displays the current weather-data for a desired location. The selection of the desired location is regulated user-dependent by a database. This software runs on a microcomputer, which also provides access to the internet. Therefore the user is able to surf the internet. All this is integrated in a framework with a built-in one-way mirror. Behind that one-way mirror is a monitor, which shows the virtual display of our microcomputer. The control of the computer happens via facial recognition and gesture-control.

Our diploma project therefore involves a **software-technical** and a **practical** part:

The **software-technical** part involves the development of an application, which determines and displays current weather-data through an API. Furthermore this weather-data should be determined based on accounts and users should be able to surf the internet. The navigation is handled by a software, which determines the commands of the user through facial recognition and gesture-control.

The source code of this diploma project is published on github and can be downloaded at <https://github.com/htl-anichstrasse/smart-mirror>.

The **practical** part involves all hardware-related criteria in this project. This involves the acquisition of the required hardware, the creation of a constructional drawing for the mirror-frame and the production of said mirror-frame.

Danksagung

An dieser Stelle möchten wir uns zunächst bei allen Personen bedanken, ohne deren Hilfe diese Diplomarbeit nicht realisierbar gewesen wäre.

Einen großen Dank möchten wir unserem betreuenden Lehrer **Herrn Prof. Mag. Dr. DI Albert Greinöcker** aussprechen, welcher uns bei Fragen und Problemen rund um diese Diplomarbeit stets mit Rat und Tat zur Seite stand. Nicht nur während der gesamten Projektdauer war er stets hilfsbereit an unserer Seite. Schon seit dem ersten Jahrgang unterrichtete er unsere Klasse und lehrte uns die Grundlagen der Softwareprogrammierung. Wir wissen ihn sowohl in fachlicher als auch in menschlicher Hinsicht sehr zu schätzen.

Des Weiteren wollen wir uns bei **Herrn Ilija Filipovic** bedanken, welcher uns seine Werkstatt trotz der schwierigen Lage mit der globalen Pandemie zur Verfügung stellte und uns bei Fragen und Problemen rund um Holzbearbeitung stets unterstützt hat. Ohne ihn wäre der Bau des Spiegelrahmens nicht möglich gewesen.

Anschließend wollten wir uns bei all unseren Freunden und Familien bedanken, welche uns während der gesamten Projektdauer stets motiviert und unterstützt haben.

Inhaltsverzeichnis

I. Intro	13
1. Ein Spiegel auf Industriestandard 4.0	14
1.1. Industrie 4.0	14
1.2. Der Spiegel	15
II. Theoretische Grundlagen	16
2. Benötigte Hardware	17
2.1. Raspberry Pi 4	17
2.1.1. Wofür einen Raspberry Pi 4	17
2.1.2. Beschaffung	17
2.1.3. Anschlüsse und Bauteile eines Raspberry Pi 4	18
2.1.4. Starten und Einrichten des Raspbian (Konfiguration)	18
2.2. Acrylglas	19
2.2.1. Beschaffung	19
2.3. Monitor	20
2.3.1. Beschaffung	20
2.3.2. Abmontage der Rückplatte	20
2.4. Webcam	21
2.4.1. Beschaffung	21
2.5. Lautsprecher	22
2.5.1. Beschaffung	22
2.6. Tastatur	23
2.6.1. Beschaffung	24
2.7. Holz	24
2.7.1. Beschaffung der Holzplatten	24
2.7.2. Beschaffung des Grundholzes	25
3. Verwendete Werkzeuge	26
3.1. Handkreissäge	26
3.2. Oberfräse	27
3.2.1. Besonderheiten einer Oberfräse	28
3.3. Kappsäge	28
3.4. Bandsäge	29
3.5. Schnitzmesser	30
3.6. Akkubohrer	32
3.7. Schleifmaschine	32

Inhaltsverzeichnis

3.8. Japansäge	33
4. Verwendete Programmiersprachen	35
4.1. Java	35
4.2. SQL	36
5. Haupttechnologien	37
5.1. CREO	37
5.2. Eclipse	37
5.3. SQLite	38
6. APIs und Libraries	39
6.1. OpenWeatherMap	39
6.1.1. API-Request	39
6.1.2. Beispiel	40
6.2. JDBC	41
III. Konstruktion des Spiegelrahmens	43
7. Modellierung	44
7.1. Einzelteile (.prt)	45
7.1.1. Bestellte Hardware	45
7.1.2. Vordere Rahmenplatte	47
7.1.3. Oberer Rahmen	48
7.1.4. Unterer Rahmen	48
7.1.5. Seitlicher Rahmen	49
7.1.6. Stabilisierungsplatte für den Monitor	50
7.1.7. Hintere Stütze für den Monitor	51
7.1.8. Hintere Rahmenplatte	51
7.2. Baugruppe (.asm)	53
7.3. Zeichnungsableitungen (.drw)	56
7.3.1. Vordere Rahmenplatte	56
7.3.2. Oberer Rahmen	57
7.3.3. Unterer Rahmen	57
7.3.4. Seitlicher Rahmen	58
7.3.5. Stabilisierungsplatte für den Monitor	58
7.3.6. Hintere Stütze für den Monitor	59
7.3.7. Hintere Rahmenplatte	59
8. Fertigung	60
8.1. Vordere Rahmenplatte	60
8.2. Oberer Rahmen	62
8.3. Unterer Rahmen	63
8.4. Seitlicher Rahmen	63
8.5. Stabilisierungsplatte für den Monitor	65
8.6. Hintere Stütze für den Monitor	66

Inhaltsverzeichnis

8.7. Hintere Rahmenplatte	67
8.8. Abschließende Schritte	69
8.8.1. Befestigung der umliegenden Rahmenteile	69
8.8.2. Befestigung der vorderen und hinteren Platte	69
8.8.3. Grundlegender interner Aufbau	70
8.8.4. Lackierung des Rahmens	71
IV. Backendstruktur	73
9. Datenbank	74
9.1. Datenbankmodell	74
9.2. SQL-Code	76
10.Implementierte Klassen	78
10.1. package database	78
10.1.1. class DBManager	78
10.1.2. class Location, User, LocationData, Alarm	96
10.2. package OpenWeatherMapAPI	99
10.2.1. class WeatherData	99
10.3. TimerTasks	102
V. Schlusswort	103
11.Aufgetretene Probleme	104
11.1. Suche der Werkstatt	104
11.2. Lieferung der benötigten Hardware	104
11.3. Fehlende Motivation	104
12.Zukünftige Schritte	106
12.1. Frontend	106
12.2. Gesichtserkennung und Gestiksteuerung	106

Teil I.

Intro

1. Ein Spiegel auf Industriestandard 4.0

1.1. Industrie 4.0

Das Zeitalter der technischen Digitalisierung rückte in den letzten Jahren immer mehr in den Vordergrund. Kennzeichen des industriellen Wandels ist die zunehmende Verwendung von intelligenten Softwaresystemen in der Produktion.

Dieser Übergang der Fertigungsprozesse und Technologien basiert unter anderem auf das Kernprinzip von **IoT**. Das sogenannte **Internet of Things** kennzeichnet die allgegenwärtige Vernetzung von Menschen, Maschinen und anderen Dingen, sowohl im physischen als auch im virtuellen Bereich. Diese Vernetzung ist in der heutigen Zeit intensiver denn je. Dies wirkt sich auf die gesamte Industrie aus. Im Produktionsprozess werden intelligente Wertschöpfungsketten entwickelt, welche die Fertigung effizienter und sparsamer machen. Nicht nur die Fertigung wird hierbei weiterentwickelt, sondern auch die Produkte, welche gefertigt werden.

Durch diese digitalisierte Fertigung konnte sich ein neuer Trend entwickeln: Die Entwicklung von Produkten mit vernetzten Softwaresystemen, welche im Grunde einem Computer ähneln. Solche Produkte werden auch **Smart-Products** genannt. *Bei Smart Products, also intelligenten Produkten, handelt es sich um solche, die flexibel an die Bedürfnisse des Anwenders anpassbar sind und mittels intelligenter Vernetzung ebenfalls mit anderen Systemen in Kontakt treten können.* [10]. Sei es bei Smart-TVs oder beim allbekannten Smart-Phone, Fakt ist, dass dieses Zeitalter der Digitalisierung schon im privaten Leben seine Spuren hinterlässt.

Wie oben bereits beschrieben wirkt sich diese Digitalisierung zunehmend auf unseren Alltag aus. Produkte, welche schon Jahrzehnte lang von uns Menschen benutzt werden, wie Handys oder Fernseher, werden durch diese industrielle Revolution immer benutzerfreundlicher und technisch fortgeschrittener. Vor Jahrzehnten konnte man sich unmöglich vorstellen mit dem eigenen Telefon oder Fernseher eine Internetverbindung aufzubauen. Heutzutage ist dies jedoch allgegenwärtig und kaum verwunderlich.

Die Welt entwickelt sich rasant weiter und die Menschheit vernetzt sich immer mehr. Wie intensiv ist diese Digitalisierung in unserer heutigen Welt? Welche Alltagsgegenstände können zu Produkten der Industrie 4.0 umgewandelt werden? Wir setzten uns als Ziel dieser Diplomarbeit eines der meist verwendeten Haushaltsgegenstände auf einen industriell modernen Stand zu bringen und einen Einblick in eine mögliche Zukunft des IoT zu erlangen. Uns war klar: Wir erstellen einen Smart-Mirror.

1.2. Der Spiegel

Wieso genau haben wir uns für die Digitalisierung eines Spiegels entschieden? Der Spiegel ist das möglicherweise meist verwendete, noch nicht allgemein digitalisierte Produkt, welches täglich von so gut wie jedem Menschen verwendet wird - ob bewusst oder unterbewusst. Gleich nach dem Aufstehen, beim Erfrischen im Bad oder nachts vor dem Schlafengehen - Wir sehen uns ständig im Spiegel.

Der Hintergedanke hier war: Wie können wir eine solch einfaches Produkt auf den Industriestandard 4.0 bringen und welche Vorteile kann der Nutzer daraus ziehen? Wir kamen schnell zu dem Entschluss, dass der Spiegel eine Anzeige haben soll, welche aktuelle Informationen abbildet. Beim Zähneputzen in der Früh wäre es doch praktisch nebenbei die aktuellen Wetterdaten einzulesen und sich schon einmal Gedanken zur heutigen Kleidungswahl zu machen. Wieso sollte man extra die aktuellen Nachrichten in einer Zeitung lesen oder im Fernseher sehen, wenn man diese doch am Badezimmerspiegel beim Vorbereiten zum Schlafengehen lesen kann?

Uns war es wichtig, dass der Nutzer durch unser Projekt eine Zeitersparnis und eine Gemütlichkeit im Alltag empfindet. Zu dieser Gemütlichkeit gehört auch die Steuerung des Spiegels. Für dieses Problem dachten wir an eine ebenfalls moderne Lösung: Gesichtserkennung und Gestiksteuerung. Beim Betreten des Bads soll der Spiegel sofort erkennen, welcher Nutzer gerade vor ihm steht, um ihm seine gewünschten Daten zu liefern. Mit der Gesichtserkennung muss der Nutzer nichts weiteres tun, als vor den Spiegel zu treten. Für die Navigation muss er, dank der Gestiksteuerung, nichts weiteres machen, als seine Hände zu bewegen.

Uns war klar, dass es sich bei diesem Projekt vergleichsweise um keine allzu große technische Innovation handelt. Jedoch wollten wir mit diesem Projekt den Grundgedanken der Digitalisierung verfolgen. Fakt ist: Die ständige industrielle Revolution betrifft immer mehr den Alltag des Menschen. Wir wollten mit unserer Diplomarbeit dieses Phänomen in einfachster Form darstellen. Wenn sogar ein Spiegel digitalisiert werden kann, kann dies so gut wie jedes Produkt.

Teil II.

Theoretische Grundlagen

2. Benötigte Hardware

2.1. Raspberry Pi 4

Ein Raspberry Pi ist ein sogenannter *Einplatinencomputer*, welcher im Gegensatz zu herkömmlichen Computern nicht allzu leistungsstark und minimalistisch gehalten ist. Im Vergleich zu den meisten Computern, welche standardmäßig auf den Betriebssystemen Windows oder MacOS laufen, besitzt der Raspberry Pi ein eigenes Betriebssystem - den sogenannten *Raspbian*, welcher auf Linux basiert. Die neuesten Modelle des Raspberry Pis befinden sich momentan in der 4. Generation. Wir erwarben für unser Projekt einen dieser neuen Modelle.

2.1.1. Wofür einen Raspberry Pi 4

Wegen seiner kleinen Rechenkapazität wird ein Raspberry Pi häufig bei Projekten eingesetzt, welche keine sonderlich hohen Rechenleistungen benötigen. Im Vergleich zu einem herkömmlichen PC kann man natürlich keine großen Leistungen erbringen, jedoch ist die Leistung des Raspberry Pi 4 konkret für unser Projekt ausreichend. Ein weiterer Aspekt, warum wir uns für die Verwendung eines Raspberry Pis entschieden haben, ist dass wir die Gesamtgröße des Spiegels bedenken mussten. Dank seiner kleinen Größe lässt sich dieser Microcomputer leicht im Inneren des Spiegelrahmens einbauen und nimmt kaum Platz ein. Zudem ist er im Vergleich zu andern Computern nicht sehr billig. Somit konnten wir die Projektkosten auf ein Minimum reduzieren.

2.1.2. Beschaffung

Wir entschieden uns für das Raspberry Pi 4 Starter-Kit von Labists. Das Produkt findet man unter folgendem Link: https://www.amazon.de/gp/product/B082HJG8S8/ref=ppx_yo_dt_b_asin_title_o02_s00?ie=UTF8&psc=1. Dieses Starter-Kit enthält:

- einen Raspberry Pi 4
- eine 32GB Micro-SD Karte mit vorinstalliertem Raspbian
- ein Micro-HDMI zu HDMI Kabel für die Verbindung mit einem Monitor
- ein USB-C Kabel mit eingebautem Einschaltknopf

Dieses Kit erwarben wir insgesamt für 109,99€.

2. Benötigte Hardware

2.1.3. Anschlüsse und Bauteile eines Raspberry Pi 4

Sieht man sich diesen Einplatinencomputer genauer an, so erkennt man die wesentlichen Anschlüsse und Bauteile (siehe Abbildung 2.1.):

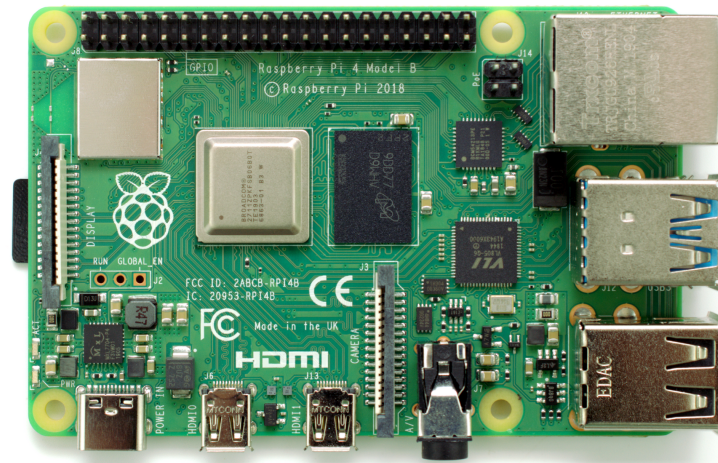


Abbildung 2.1.: Raspberry Pi 4

- RAM (schwarze Steckerleiste oben links)
- CPU (Metallplatte rechts neben dem Raspberry Pi Logo)
- USB-C Anschluss zur Stromversorgung (unten links)
- 2x Micro-HDMI Anschlüsse (die zwei Anschlüsse neben dem USB-C Anschluss)
- Gigabit-Ethernet Anschluss (oben rechts)
- 2x USB 3.0 Anschlüsse (direkt unter dem Gigabit-Ethernet Anschluss)
- 2x USB 2.0 Anschlüsse (unten rechts)
- Micro-SD Anschluss (ganz links auf der Unterseite)

Zudem wird ein Lüfter mitgeliefert, welcher die CPU je nach Anschluss im leisen Kühlmodus (schwächer) oder im Vollgeschwindigkeits-Kühlmodus (stärker) abkühlt. Für unsere Zwecke haben wir den Vollgeschwindigkeits-Kühlmodus gewählt.

2.1.4. Starten und Einrichten des Raspbian (Konfiguration)

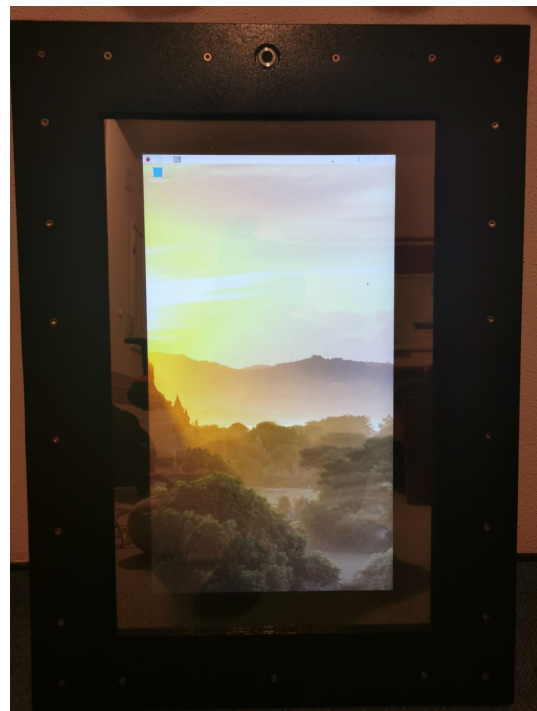
Die mitgelieferte Micro-SD-Karte hatte den Raspbian schon vorinstalliert. Sobald man diese in den Anschluss steckt und den Microcomputer startet, wird das System automatisch installiert. Dann gilt es nur noch ein Profil zu erstellen und eine WLAN-Verbindung einzurichten.

2.2. Acrylglas

Ein Grundproblem bei unserer Diplomarbeit war die Spiegelscheibe. Wir benötigen eine Scheibe, welche einerseits das Spiegelbild reflektiert und andererseits das Bild des Monitors erkenntlich hindurch projizieren lässt. Unser erster Instinkt für die Lösung dieses Problems war die Verwendung eines Einwegspiegels. Diese sind jedoch einerseits teuer und schwer zu finden. Deshalb überlegten wir uns eine billigere Alternative: Wir erwerben eine komplett transparente Acrylglasscheibe und kleben eine halbreflektierende Spiegelfolie auf eine Seite. Der resultierende Effekt ist der selbe wie bei einem Einwegspiegel (siehe Abbildung 2.2.).



(a) Reflexionseffekt Acrylglas



(b) Durchlasseffekt Acrylglas

Abbildung 2.2.: Einwegspiegeleffekt mit Acrylglas

2.2.1. Beschaffung

Hier erwarben wir eine völlig transparente Acrylglasscheibe mit den Dimensionen 600mm x 400mm x 3mm für 23,70€. Diese Scheibe findet man unter folgendem Link: https://www.amazon.de/gp/product/B08B8TQ7XQ/ref=ppx_yo_dt_b_asin_title_o00_s00?ie=UTF8&psc=1.

Zudem bestellten wir eine klebende Spiegelfolie mit den Dimensionen 44,5cm x 200cm für 16,40€ unter folgendem Link: https://www.amazon.de/gp/product/B08G4S28ZK/ref=ppx_yo_dt_b_asin_title_o00_s00?ie=UTF8&psc=1.

2. Benötigte Hardware

2.3. Monitor

Bei der Beschaffung eines Monitors mussten wir abgesehen vom Preis auf keine Kriterien achten. Das Einzige was dieser Monitor benötigt ist ein LCD-Display und einen HDMI Anschluss. Beides sind Kriterien, welche heutzutage auf so gut wie alle Monitore zutreffen.

Wichtig ist hierbei, dass die Rückplatte des Monitors abmontiert werden muss, da diese ansonsten zu viel Platz beim Zusammenbau wegnehmen würde. Zudem dient die Innenplatte des Monitors als gute Grundfläche, um andere Bauteile, wie z.B.: den Raspberry Pi zu montieren.

2.3.1. Beschaffung

Beim Kauf des Monitors entschieden wir uns für den AOC 22B1H Monitor (siehe Abbildung 2.3.), da dieser keine gravierenden Mängel aufweist und im Vergleich zu ähnlichen Produkten relativ billig ist (83,45€).

Diesen Monitor kann man unter folgendem Link finden:

https://www.amazon.de/gp/product/B07CFS5TBF/ref=ppx_yo_dt_b_asin_title_o01_s00?ie=UTF8&psc=1.



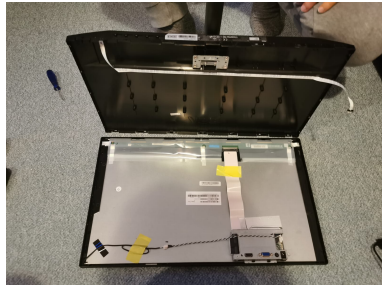
Abbildung 2.3.: AOC 22B1H Monitor

2.3.2. Abmontage der Rückplatte

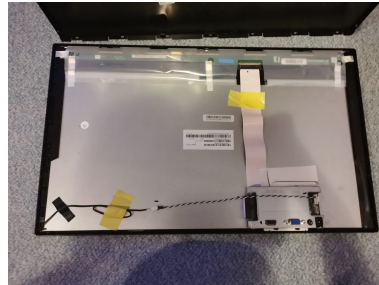
Bei der Abmontage der Rückplatte musste diese einfach mit einem Schraubenzieher ausgehebelt werden. Dadurch lösen sich die Steckverbindungen zwischen Rück- und

2. Benötigte Hardware

Vorderplatte nach und nach. Nachdem diese gelöst wurden lässt sich die Rückplatte einfach entfernen (siehe Abbildung 2.4.).



(a) Innenseite



(b) Innenseite

Abbildung 2.4.: Innenseite eines Monitors

Nachdem die Rückplatte entfernt wurde kann auch eine grundlegende Positionierung des Raspberry Pis gemacht werden (siehe Abbildung 2.5.).

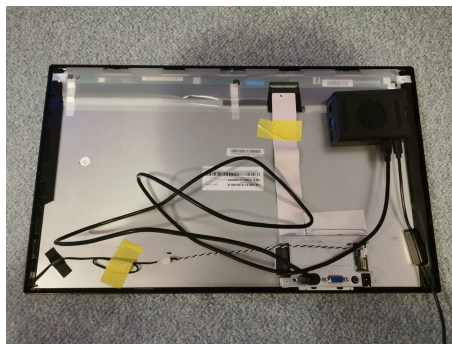


Abbildung 2.5.: Grundlegende Positionierung auf der Innenplatte des Monitors

2.4. Webcam

Bei der Webcam für die Gesichtserkennung und Gestiksteuerung mussten wir nicht auf allzu viele Kriterien achten. Wichtig war, dass die Webcam mit dem USB 3.0 Anschluss des Raspberry Pi kompatibel ist. Zudem war es für den Zusammenbau mit dem Spiegelrahmen wichtig, dass die Linse etwas hinausragt, damit sie sich auf dem oberen Spiegelrahmen abstützen kann.

2.4.1. Beschaffung

Wir fanden eine relativ billige Webcam (14,99€), welche damit wirbt mit dem Raspberry Pi kompatibel zu sein. Diese Webcam ist die Sunflower Bag Raspberry Pi USB 2.0 Free Driver Camera (https://www.amazon.de/-/en/gp/product/B078Y5N9X2/ref=ppx_yo_dt_b_asin_title_o00_s00?ie=UTF8&psc=1).

2. Benötigte Hardware

Da ihre Linse ein wenig herausragt ist sie für unseren Zusammenbau ideal geeignet und sie besitzt sogar eine flexible Kabellänge (siehe Abbildung 2.6.).



Abbildung 2.6.: Sunflower Bag Raspberry Pi USB 2.0 Free Driver Camera

2.5. Lautsprecher

Genau wie bei dem Monitor und bei der Webcam hatten wir auch bei den Lautsprechern nicht nach allzu hohe Anforderungen. Uns war wiederum wichtig, dass sich die Lautsprecher einfach in einen Spiegelrahmen integrieren lassen (einfache Form) und dass wir eine möglichst billige Variante finden.

2.5.1. Beschaffung

Wir entschieden uns für die Audiocore AC870 Stereo Speakers (https://www.amazon.de/-/en/gp/product/B01LCXFEMM/ref=ppx_yo_dt_b_asin_title_o00_s00?ie=UTF8&psc=1). Diese bieten trotz ihrem unglaublich billigen Preis (8,56€) eine völlig ausreichende Qualität für unseren Zweck. Zudem haben sie eine einfache Form, sodass diese einfach in den seitlichen Rahmen eingebaut werden können. Dank ihrem USB-Anschluss lassen sie sich einfach an den Raspberry Pi anschließen (siehe Abbildung 2.7).

2. Benötigte Hardware



Abbildung 2.7.: Audiocore AC870 Stereo Speakers

Zu beachten ist jedoch, dass sie einen leichten Rauscheffekt aufweisen. Allerdings ist dies für unsere Zwecke nicht von allzu hoher Bedeutung.

2.6. Tastatur

Die endgültige Navigation auf dem Spiegel soll letztendlich über Gesichtserkennung und Gestiksteuerung funktionieren. Bis dahin benötigen wir eine weitere Möglichkeit für die Navigation. Wichtig ist, dass diese Navigationsmöglichkeit kabellos funktioniert, da außer den Stromkabeln keine anderen Kabel aus dem Spiegelrahmen hinausschauen sollen.

Wir haben uns daher für eine kabellose Tastatur mit integriertem Mauspad entschieden. Für diese Tastatur gibt es einen USB-Adapter, welcher in den Raspberry Pi angesteckt werden muss. Über diesen kann die Tastatur kabellos mit dem Raspberry Pi verbunden werden.

2. Benötigte Hardware

2.6.1. Beschaffung



Abbildung 2.8.: Logitech Wireless Keyboard

Explizit entschieden wir uns für das Logitech Wireless Keyboard, welches wir für 20,68€ erwarben. Finden kann man diese Tastatur unter folgendem Link: https://www.amazon.de/-/en/gp/product/B00VHHWNMI/ref=ppx_yo_dt_b_asin_title_o00_s00?ie=UTF8&psc=1. Diese Tastatur kommt mit integriertem Mauspad, weshalb wir uns ein ganzes Bauteil ersparen.

2.7. Holz

Das vermutlich wichtigste Element unseres Spiegels ist der Holzrahmen. Folglich war es auch wichtig ein sowohl stabiles, als auch einfach zu bearbeitendes Holz zu beschaffen. Insgesamt benötigten wir drei Platten (vorne, hinten und Stabilisierungsplatte des Monitors) und ein Grundholz, um die restlichen Elemente des Rahmens zu fertigen.

2.7.1. Beschaffung der Holzplatten

Die Stabilisierungsplatte des Monitors mussten wir nicht kaufen, da eine passende Platte schon in unserem Besitz war. Die Platte musste mindestens die Dimensionen 600mm x 400mm x 10mm besitzen. Wir hatten eine Platte mit den Dimensionen 700mm x 450mm x 10mm.

Bei der vorderen und hinteren Platte benötigten wir die Dimensionen 730mm x 530mm x 10mm. Da beide Platten die gleiche Dimension hatten beschafften wir 2 Platten mit der gleichen Größe beim OBI in Wörl. Diese Platten wurden beim Kauf direkt auf die benötigten Maße zugeschnitten. Zu beachten ist hierbei jedoch, dass die Platten nicht eine Dicke von 10mm aufweisen, sondern eine Dicke von 8mm. Für unsere Zwecke ist dies jedoch kein allzu großer Unterschied und ändert nichts am Endprodukt.

2. Benötigte Hardware

Bei allen oben genannten Platten handelt es sich um sogenannte MDF-Platten. *Die mitteldichte Holzfaserplatte oder mitteldichte Faserplatte, kurz MDF-Platte, ist ein Holzwerkstoff aus dem Bereich der Faserplatten.* [15]. Diese MDF-Platten sind sowohl sehr stabil, als auch leicht zu bearbeiten, weshalb sie für unseren Zweck ideal sind.

2.7.2. Beschaffung des Grundholzes

Bei dem Grundholz für alle anderen Holzelemente entschieden wir uns für Fichtenholz. Dieses ist sowohl relativ weich (leicht zu bearbeiten) als auch sehr stabil.

Wir benötigten insgesamt ein Grundmaß von 62mm x 65mm und variierender Länge. Insgesamt beschafften wir uns 5 gleich große Holzpfiler, welche wir dann auf Form zuschnitten. Diese Holzpfiler haben jeweils die Dimensionen 90mm x 90mm x 1000mm. Beschafft wurden diese bei der Neuschmied-Tischlerei in Hopfgarten im Brixental.

3. Verwendete Werkzeuge

3.1. Handkreissäge

Das wohl wichtigste Werkzeug bei der Holzbearbeitung ist die Handkreissäge (siehe Abbildung 3.1.). Mit ihr ist es möglich lange, gerade Schnitte in Massivholz oder Holzwerkstoffplatten anzufertigen. *Mit ihrem schnell rotierenden Sägeblatt frisst sich die Handkreissäge präzise durch das zu schneidende Material.* [7].



Abbildung 3.1.: Handkreissäge

Ein sauberer Schnitt setzt voraus, dass die Handkreissäge über eine elektronische Drehzahlstellung verfügt, welche die Umdrehungsgeschwindigkeit des Sägeblatts auch unter Last konstant hält.

newline

Insgesamt unterscheidet man zwischen 2 Arten von Handkreissägen:

- Die **Pendelhaubensäge**, bei der das Sägeblatt unten immer fest heraussteht. Die Schutzhaube wird beim Sägen vom Werkstück zurückgedrückt und pendelt beim

3. Verwendete Werkzeuge

Hochnehmen wieder zurück.

- Die **Tauchsäge**, bei der das Sägeblatt senkrecht heruntergedrückt wird. Diese Säge "taucht" somit ab.

In unserem Fall wurde eine Tauchsäge verwendet, welche uns zudem auch ermöglichte mitten im Werkstoff einen Schnitt zu setzen (statt an einer Kante).

3.2. Oberfräse

Eine Oberfräse (siehe Abbildung 3.2.) dient zum Fräsen von Holzwerkstücken. Mit ihr können komplizierte Formen im Werkstück auf einfache Art realisiert werden.

An kein Elektrowerkzeug werden so hohe Ansprüche an die Genauigkeit gestellt wie an die Oberfräse. Sie kommt zum Einsatz, wenn besonders präzise und saubere Ergebnisse gewünscht sind. [6].



Abbildung 3.2.: Oberfräse

3. Verwendete Werkzeuge

Hierbei liegt das Werkzeug, genau wie auf der Abbildung 3.2. zu sehen ist, auf dem Werkstück auf. Die Länge des Fräasers, welcher sich in der Mitte befindet, kann über die linke Skala eingestellt werden. Sobald die gewünschte Länge gewählt wurde, kann parallel zur aufliegenden Kante gefräst werden.

In unserem Fall wurde die Oberfräse vor allem bei der Realisierung von gewissen Schlitzten für Kabel und rechteckige Ausnehmungen verwendet.

3.2.1. Besonderheiten einer Oberfräse

- Die Unterseite der Oberfräse besitzt eine Kunststoffschicht, welche schonend über die aufliegende Fläche des Werkstücks gleitet.
- Anhand des hochtourig laufenden Motors (ca. 24.000 U/min) fallen sehr feine Späne an. Deshalb ist es wichtig eine funktionierende Staubabsaugung anzuschließen.
- Bei unterschiedlichen Werkstoffen sind unterschiedliche Fräsgeschwindigkeiten zu wählen. Zudem soll die Drehzahl mit steigendem Fräserdurchmesser abnehmen.

3.3. Kappsäge

Eine Kappsäge kommt ermöglicht das Kappen (Kürzen) von Holz. Durch sie ist es also möglich das Holz auf die benötigten Längen zu kürzen. Sieht man sich den Aufbau der Kappsäge an, wird auch klar wie diese funktioniert (Abbildung 3.3.):



Abbildung 3.3.: Aufbau einer Kappsäge

3. Verwendete Werkzeuge

Das Holz wird hierbei auf dem Sägetisch entlang der Anschlagsschiene platziert. Nach Einschalten des Motors rotiert das Sägeblatt und das Holz kann im rechten Winkel gekürzt werden. Hierbei muss das Sägeblatt nach Einschalten des Motors über den Griff senkrecht nach unten geführt werden. Daraufhin kann das Sägeblatt über die Zugfunktion auch vertikal verschoben werden, um in das Werkstück hinein zu schneiden.

Zu Beachten ist, dass man beim Anlegen des Werkstücks auch noch die Breite des Sägeblattes mit einzuberechnen ist. Die Kappsäge, welche von uns verwendet wurde, besaß eine Sägeblatt-Breite von 7mm. Somit musste das Werkstück weitere 7mm von unserem Schnittpunkt entfernt aufgelegt werden.

Hauptsächlich benutzten wir die Kappsäge um die 5 Holzpfiler (90mm x 90mm x 1000mm) auf die richtige Länge zu kürzen. Somit gelang durch die Kappsäge eine Grundlage für die nachfolgende Bearbeitung zu schaffen.

3.4. Bandsäge

Eine Bandsäge ist ein Werkzeug zur Holzbearbeitung, welche aus einem in sich geschlossenem Sägeblatt besteht. Daher kommt auch der Name, da das Sägeblatt ein endloses Band ist.

Am Sägetisch kann die Anschlagsschiene wiederum verstellt werden. Sollte der Abstand zum Sägeblatt richtig justiert sein kann der Motor gestartet und die Maschine in Betrieb genommen werden. Durch die parallele Positionierung der Anschlagsschiene zum Sägeblatt lässt sich ein paralleler Schnitt ermöglichen.

Die von uns benutzte Bandsäge (siehe Abbildung 3.4) wurde deshalb hauptsächlich dazu verwendet die Holzpfiler und Holzplatten auf die richtigen Grundmaße zu schneiden.

3. Verwendete Werkzeuge



Abbildung 3.4.: Bandsäge

3.5. Schnitzmesser

Schnitzmesser (siehe Abbildung 3.5.) zeichnen sich vor allem durch ihre außerordentliche Schärfe aus. Falls genaue Formen benötigt werden bzw. falls eine geringe Menge Holz entfernt werden muss ist die Verwendung eines Schnitzmessers eine gute Idee.

3. Verwendete Werkzeuge



Abbildung 3.5.: Schnitzmesser

Oft wird ein Schnitzmesser bei der Holzbearbeitung in Kombination mit einem Schonhammer (siehe Abbildung 3.6.) eingesetzt. Hierbei wird das Schnitzmesser in Position gebracht und dann mit dem Schonhammer auf die Unterseite des Messers geschlagen. Somit wird ein Kraftstoß des Schnitzmessers gewährleistet und Material entfernt.



Abbildung 3.6.: Schonhammer

In unserem Fall wurde das Schnitzmesser vor allem für die Realisierung von rechteckigen Formen verwendet. Auch falls unerwünschte Überreste von anderen Bearbeitungen vorhanden waren konnten diese mit dem Schnitzmesser entfernt werden.

3. Verwendete Werkzeuge

3.6. Akkubohrer

Ein Werkzeug, welches bei der Holzbearbeitung nicht vergessen werden darf, ist der Akkubohrer. Dieser dient in unserem Fall vor allem für die Setzung von Verbindungen zwischen zwei Holzstücken.



Abbildung 3.7.: Akkubohrer

Mit diesem Akkubohrer wurden in unserem Fall zwei Aktionen durchgeführt:

- Bohren von Löchern für eine Dübelverbindung zwischen zwei Holzstücken
- Bohren von Schraubverbindungen

3.7. Schleifmaschine

Mit der Schleifmaschine gelang uns ein einfaches Abschleifen aller zuvor bearbeiteten Flächen.

3. Verwendete Werkzeuge



Abbildung 3.8.: Schleifmaschine

In dem von uns benutzten Modell wurde auf der Unterseite ein rundes Schleifpapier eingespannt. Nach Einschalten der Maschine beginnt dieses zu rotieren und ermöglicht somit ein effizientes Schleifen der Oberfläche.

3.8. Japansäge

Eine Japansäge ist eine Handsäge, welche im Gegensatz zu anderen Sägen nur auf Zug arbeitet. Zu erkennen ist, dass hier für diesen Zugschneideffekt die Zähne zum Griff zeigen.

3. Verwendete Werkzeuge



Abbildung 3.9.: Japansäge

Während auf Druck wirkende Sägen ein relativ breites und starres Blatt benötigen und sich trotzdem verbiegen können, ermöglicht das Zugprinzip der japanischen Sägen ein dünneres Sägeblatt (0,3–0,6 mm, bei Zimmermannssägen auch bis ca. 1 mm), was sehr feine Schnitte mit geringem Kraftaufwand erlaubt. [13]

Mit der Japansäge wurden in unserem Fall hauptsächlich Schnitte, welche mit der Handkreissäge gemacht wurden, fein auf Form zugeschnitten. Überreste, welche nicht von der Handkreissäge abgetragen wurden, wurden mit dieser Säge entfernt.

4. Verwendete Programmiersprachen

4.1. Java

Bei der Wahl der Programmiersprache legten wir den Hauptfokus darauf, unsere Software-Anwendung nicht nur auf den Raspberry-Pi, welchen wir für den Prototyp ausgewählt haben, zu optimieren, sondern dass diese auch ohne große Anpassungen auf möglichst vielen verschiedenen Endgeräten ohne zusätzliche Softwarepakete funktionieren soll. Dadurch sind plattformabhängige Programmiersprachen ausgeschlossen.

Möglich wäre hierbei auch die Verwendung von Python. Typischerweise wird diese Programmiersprache auch bei der Verwendung eines Raspberry Pis benutzt. Wir entschieden uns jedoch für Java, da wir diese Sprache in unserer Ausbildung intensiver verwendet haben und vertrauter damit sind.

Für das Front- und Backend-Development haben wir uns daher für die plattformunabhängige, objektorientierte Programmiersprache Java entschieden. Java ist auf den meisten infrage kommenden Endgeräten bereits verfügbar, während viele Funktionen ebenfalls mit wenig Aufwand über bereits vorhandene Klassen umsetzbar sind.

Die Java-Technologie umfasst die Programmiersprache Java, als auch das Java Development Kit **JDK** und das Java Runtime Environment **JRE**. JDK ist das Entwicklungswerkzeug und enthält einen Compiler der den Quellcode in einen Bytecode umwandelt, welcher anschließend über das am Zielsystem installierte JRE in die jeweils richtige Maschinensprache interpretiert wird. [14].

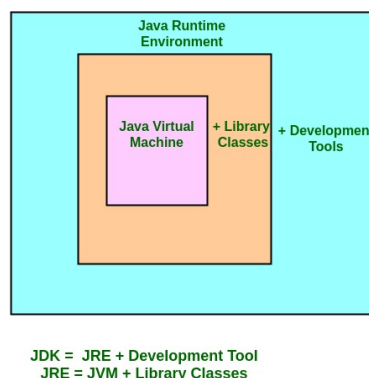


Abbildung 4.1.: Aufbau der Java-Technologie

4. Verwendete Programmiersprachen

Dank der Objektorientierung und Plattformunabhängigkeit ist Java heutzutage eine der am weitesten verbreiteten Programmiersprachen für Anwendungen auf PCs und Smartphones, aber auch für Server und Backend-Logiken von vielen Webseiten. Da ein Teil des Programmcodes über die Laufzeitumgebung JRE erst übersetzt wird, lassen sich die Hauptteile der Anwendungen kompakt halten und es wird relativ wenig Speicherplatz für das Haupt-Programm selbst benötigt [5].

4.2. SQL

SQL steht für Structured Query Language und ist eine der Standard-Programmiersprachen wenn es um relationale Datenbanken geht. Viele andere Datenbanksprachen wie z.B. Microsoft-SQL, MySQL und Oracle basieren auf SQL und bieten noch einige Erweiterungen, bringen aber auch zusätzliche Anforderungen bzw. Einschränkungen mit sich, wie Microsoft-SQL die Abhängigkeit zum Betriebssystem [1].

Eine Datenbanksprache wird benötigt um Datenbanken zu erstellen, mit Daten zu füttern, diese zu verändern und abfragen zu können. SQL ist eine sehr einfach gehaltene Datenbanksprache und basiert auf der relationalen Algebra. Es gibt drei Kategorien von SQL Befehlen [11]:

- **DDL (Data Definition Language):** Anlegen/Löschen/Ändern der Datenbank-Tabellen
- **DCL (Data Control Language):** Zugriffsberechtigung der generierten Datenbank-Tabellen
- **DML (Data Manipulation Language):** Einfügen/Löschen/Lesen von Daten aus den Datenbank-Tabellen

Für gewöhnlich werden die notwendigen Datenbank-Tabellen initial über DDL erstellt, und von da an nur mehr über DML Inhalte den Tabellen hinzugefügt, bzw. gelesen/verändert.

In Ausnahmefällen bzw. bei komplexeren Anwendungen kann es auch während der Laufzeit notwendig sein Tabellenstrukturen und Berechtigungen anzupassen. SQL ist daher nicht als vollwertige Programmiersprache anzusehen, da mit SQL-Befehlen allein keine kompletten Programme geschrieben werden können. Viel mehr verwenden andere Programmiersprachen SQL-Befehle um Datenbank-Operationen durchzuführen.

5. Haupttechnologien

5.1. CREO

Creo Elements/Pro ist eine parametrische 3D-CAD-Software, auch bekannt unter dem Namen Pro/ENGINEER, Pro/E oder ProE. Hersteller dieser Software ist die Parametric Technology Corporation (PTC). [8].

Mit Creo können 3D-Modelle aller Art erstellt werden. Aus diesen 3D-Objekten können anschließend Baugruppen (.asm) und Zeichnungsableitungen (.drw) erstellt werden. Durch die Baugruppen kann erkannt werden in wie Fern die einzelnen Bauteile zusammenpassen. Die Zeichnungsableitungen stellen das 3D-Objekt auf einer 2D-Zeichnung in den 3 benötigten Ansichten dar.

Das erste Konzept unseres Spiegelrahmens wurde mittels Creo erstellt. Durch die gesammelte Erfahrung mit dieser Software in den ersten Jahrgängen war der Umgang mit dem Programm nicht schwer. Zuerst erstellten wir für jedes bestellte Hardware ein 3D-Modell. Schließlich folgte die Erstellung der einzelnen Holzrahmentteile und eine Baugruppe in der all diese Einzelteile zusammengebaut wurden. Hierbei ließ sich auch erkennen, ob das Konzept weiter verbessert werden konnte. Schließlich wurden aus allen Holzrahmentteilen jeweils eine Zeichnungsableitung erstellt, mit der die Fertigung in der Werkstatt einfach durchzuführen war.



Abbildung 5.1.: Creo Logo

5.2. Eclipse

Um unsere Software-Anwendung in Java zu schreiben würde man theoretisch nicht mehr benötigen als das Java Development Kit **JDK**, das Java Runtime Environment **JRE** am Zielsystem und einen simplen Texteditor bei kleineren Projekten. Bei dem Endgerät, auf dem die Software ausgeführt werden soll ist sogar nur die JRE allein erforderlich [3].

5. Haupttechnologien

Für die Erstellung des Programmcodes haben wir uns für die Entwicklungsumgebung **Eclipse** entschieden, die viele Funktionen für die Java-Programmierung mitliefert. Eclipse ist eine open-source IDE (integrierte Entwicklungsumgebung) und wird primär für die Java-Programmierung benötigt, kann aber auch für andere Programmiersprachen verwendet werden [12]. Geschriebene Programme können in Eclipse direkt ausgeführt werden, und z.B. durch automatische Quelltextvervollständigung wird vielen Fehlern vorgebeugt.



Abbildung 5.2.: Eclipse Logo

5.3. SQLite

Das Problem bei vielen Datenbanken, wie Oracle oder MySQL ist, dass diese sehr viel Speicherplatz einnehmen und diese einen Server benötigen, weshalb der Nutzer diese Technologien selbstständig installieren muss. Diese Datenbanksysteme haben zwar ihre Vorteile, sind allerdings für kleinere Projekte, wie unseren Smart-Mirror, viel zu umfangreich.

Die Desktop-Datenbank SQLite besteht aus einer Library, welche unter Java einfach integriert werden kann. Der Nutzer des Programms muss hierbei keine andere Software installieren, da es sich bei SQLite um eine Desktop-Datenbank handelt. Das bedeutet, dass diese Datenbank die Zugriffe alleine verwaltet und diese nicht von einem Server verwaltet werden (siehe Abbildung 5.3) [2].

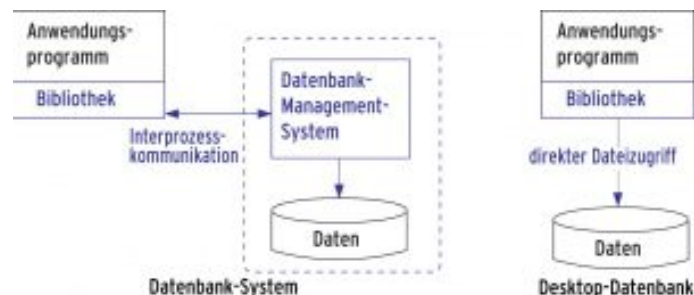


Abbildung 5.3.: Unterschied Datenbanksystem vs. Desktop-Datenbank

Das Anwendungsprogramm liest und schreibt bei einer Desktop-Datenbank direkt, während bei einem Datenbanksystem vorher ein Server gestartet werden muss. Die einzelnen Datenbanken werden mit SQLite jeweils in einer .db-Datei gespeichert.

6. APIs und Libraries

6.1. OpenWeatherMap

Da wir für unseren Spiegel das aktuelle Wetter anzeigen wollen, müssen wir aus irgendeiner Quelle diese benötigten Daten beziehen. Nach einiger Recherche fanden wir die OpenWeatherMap-API.

Bei der OpenWeatherMap-API handelt es sich um eine Schnittstelle, welche auf Anfrage die aktuellen Wetterdaten aus ihrer Datenbank bezieht und in Form einer JSON-Antwort an den Nutzer sendet. Mit dieser API ist es möglich neben den aktuellen Daten auch Daten für eine Wettervorhersage zu beziehen. [4].

6.1.1. API-Request

Bezogen werden können diese Daten über verschiedene Parameter:

- Name der Stadt
- Stadt-ID (cityID) in der Datenbank von OpenWeatherMap
- Geografische Koordinaten
- Postleitzahl

Einer dieser Werte muss für die Anfrage in die URL geschrieben werden. Zudem wird ein API-Key benötigt, welcher beim Erstellen eines kostenlosen Kontos zugesendet wird. Dieser muss ebenfalls in der URL angegeben werden.

OpenWeatherMap bietet auf ihrer Webseite eine JSON-Datei mit Werten von allen Städten an, mit denen ein API-Request möglich aus. Bei dieser Datei handelt es sich um die Datei **city.list.json**, welche unter <https://bulk.openweathermap.org/sample/> heruntergeladen werden kann. Wir lesen diese Datei zu Beginn unseres Programms ein und speichern die wesentlichen Werte (z.B.: cityID) in unsere lokale Datenbank. Unser API-Call erfolgt somit über die cityID.

6.1.2. Beispiel

Eine Beispiel URL für das Beziehen von den aktuellen Wetterdaten von Innsbruck wäre: **`http://api.openweathermap.org/data/2.5/weather?id=2775220&appid=[API-KEY]&units=metric`**. Bei dem Parameter **id** handelt es sich hierbei um die cityID von Innsbruck. Der Parameter **appid** gleicht dem API-Key. Zudem wird angegeben, dass die Einheiten in metrischer Form angegeben werden sollen (**units=metric**).

Die JSON-Antwort der API sieht folgendermaßen aus:

```

1 {
2   "coord": {
3     "lon": 11.3945,
4     "lat": 47.2627
5   },
6   "weather": [
7     {
8       "id": 501,
9       "main": "Rain",
10      "description": "moderate rain",
11      "icon": "10n"
12    }
13  ],
14  "base": "stations",
15  "main": {
16    "temp": 7.25,
17    "feels_like": 3.11,
18    "temp_min": 6.11,
19    "temp_max": 8,
20    "pressure": 1011,
21    "humidity": 71
22  },
23  "visibility": 5000,
24  "wind": {
25    "speed": 3.6,
26    "deg": 20
27  },
28  "rain": {
29    "1h": 0.37
30  },
31  "clouds": {
32    "all": 75
33  },
34  "dt": 1615502293,
35  "sys": {
36    "type": 1,
37    "id": 6872,
38    "country": "AT",
39    "sunrise": 1615440925,
40    "sunset": 1615482818
41  },
42  "timezone": 3600,
43  "id": 2775220,
44  "name": "Innsbruck",
45  "cod": 200
46 }
```

Folgende Werte sind für unser Projekt von Bedeutung:

- **description:** Eine kurze, textliche Beschreibung der aktuellen Wetterlage.
- **icon:** Ein Bild, welches die aktuelle Wetterlage darstellt.
- **temp:** Die aktuelle Temperatur.
- **humidity:** Die aktuelle Luftfeuchtigkeit.
- **wind:** Wert für das aktuelle Windverhalten.
- **timezone:** Die Zeitverschiebung zur UTC (koordinierte Weltzeit) in Sekunden.

Mit all diesen Werten kann im Programm eine Darstellung bzw. Weiterverarbeitung gemacht werden.

6.2. JDBC

Um unsere Datenbank mit dem Java-Code zu verbinden und von ihm aus Zugriff auf die Datenbank zu erhalten benötigen wir eine Schnittstelle. Die Library JDBC (**J**ava **D**ata**B**ase **C**onnection) bietet sich hierbei perfekt an, da wir mit dieser schon in unserer HTL-Ausbildung gearbeitet haben und diese leicht zu implementieren ist.

Nach der Einbindung der Library in das Projekt muss als erster Schritt ein Connection-Objekt erzeugt werden. Dieses ist, wie der Name schon verrät, für die Verbindung mit der Datenbank verantwortlich.

```
1 Connection con = DriverManager.getConnection([Dateipfad]);
```

Diese Verbindung wird über einen DriverManager erzeugt. Dieser ist für jegliche Datenbanktreiber-Aktionen verantwortlich und wird in unserem Fall nur für den Verbindungsaufbau verwendet.

Als Parameter muss hier der Dateipfad der SQLite Datenbank als String angegeben werden, damit auch klar ist, mit welcher Datenbank die Verbindung aufgebaut werden soll.

Sobald die Verbindung aufgebaut wurde, können über diese SQL-Befehle (= Statement bzw. PreparedStatement) ausgeführt werden. Ein String, welcher einen SQL-Befehl enthält, kann sehr leicht in ein Statement- bzw. PreparedStatement-Objekt umgewandelt werden:

```
1 PreparedStatement stm = con.prepareStatement([SQL-Befehl]);
```

Variable Parameter im SQL-Syntax können beim preparedStatement-Befehl mit ? gekennzeichnet werden und im Nachhinein mit folgendem Befehl befüllt werden:

```
1 stm.set[DATATYPE]([POSITION DES ?], [VALUE]);
```

Beispielsweise sieht eine solche Befüllung in unserem Fall so aus:

```
1 stm.setInt(1, l.getLocationID());
```

6. APIs und Libraries

Hier wird das erste ?, bei dem es sich um einen Integer handelt, mit dem Wert `l.getLocationID()` befüllt. Sobald alle notwendigen Werte befüllt wurden, kann der Befehl ausgeführt werden.

Beim Ausführen des Statements gibt es mehrere Möglichkeiten:

- **`stm.executeUpdate()`**: Verändert einen oder mehrere Datensätze. Dies kann beispielsweise in Form von UPDATE oder INSERT Befehlen erfolgen.
- **`stm.executeQuery()`**: Verändert die Datensätze nicht, sondern gibt diese nur zurück. Typischerweise wird dies bei SELECT Befehlen verwendet.

Im letzteren Fall kann man den Rückgabewert in ein ResultSet-Objekt speichern. Dieses ResultSet enthält alle angeforderten Werte des SQL-Befehls. Diese Werte können über die Position ihres Rückgabewerts mit dazugehörigem Datentyp erhalten werden.

Beispiel:

```
1 String sql = "select locationID, location from location, where locationID = 1;";
2 PreparedStatement stm = con.prepareStatement(sql);
3 ResultSet rs = stm.executeQuery();
4 int i = rs.getInt(1);
```

In der Variable `i` wird somit der Wert des ersten angeforderten Parameters gespeichert, also die `locationID`.

Teil III.

Konstruktion des Spiegelrahmens

7. Modellierung

Vor der Fertigung ist es wichtig zu wissen, wie die einzelnen Elemente des Spiegelrahmens aussehen müssen. Hierbei bietet sich eine grundlegende, virtuelle Modellierung dieser Bauteile an, da hier leicht Veränderungen an der Grundstruktur vorgenommen werden können und das Zusammenspiel der einzelnen Bauteile überprüft und optimiert werden kann.

Grundlegend benötigen wir für den Holzrahmen:

- eine Oberseite
- eine Unterseite
- seitliche Rahmenteile
- eine Vorderplatte
- eine Rückplatte

Welche Maße diese Einzelteile haben sollen, wird dann über das Modell erkenntlich.

Für die Modellierung haben wir das 3D-Modellierungsprogramm Creo Parametric 4.0 verwendet, da wir mit diesem Programm schon in den ersten Jahren unserer Ausbildung reichlich Erfahrung gesammelt haben.

Mit Creo können die einzelnen Bauteile als 3D-Modell erstellt werden. Hierbei spricht man von einer .prt (Part) – Datei. Mehrere Bauteile können dann in einer Baugruppe (.asm) zusammengefügt werden und anschließend kann für jedes Bauteil eine Zeichnungsableitung in den drei Ansichten abgeleitet werden (.drw). Anhand dieser Zeichnungsableitungen kann anschließend der endgültige Spiegelrahmen in der Werkstatt gefertigt werden.

Bei der Modellierung sind wir grundsätzlich folgendermaßen vorgegangen:

1. Grundlegende Modellierung eines Einzelteils
2. Einfügen des modellierten Einzelteils in der Baugruppe
3. Überprüfen, ob das Einzelteil in Abhängigkeit zu den bereits vorhandenen Einzelteilen passt
4. Evtl. Optimieren des Einzelteils

Sobald das Einzelteil in Ordnung ist, wird aus diesem eine Fertigungszeichnung abgeleitet.

7.1. Einzelteile (.prt)

7.1.1. Bestellte Hardware

Da der erste Schritt des Praxisteils die Bestellung der notwendigen Hardware war, befand sich diese vor der Modellierung in unserem Besitz. Grundlegend wurde als erster Schritt jedes einzelne Hardware-Element und deren Zusammenspiel getestet.

Sobald feststand, dass diese in Ordnung sind und wir sie für unseren Smart-Mirror benutzen werden, mussten wir sie für die Fertigung irgendwie ins Modell einbinden. Sie dienen somit als Grundlage für die Modellierung des Holzrahmens.

Es war also klar: Als erster Schritt muss die Hardware als .prt-Datei im Creo-Verzeichnis modelliert werden. Hierbei wurden die benötigten Maße mit einem Messschieber abgemessen und anhand dieser das Bauteil modelliert. Die wesentliche Hardware für die Modellierung beinhaltet:

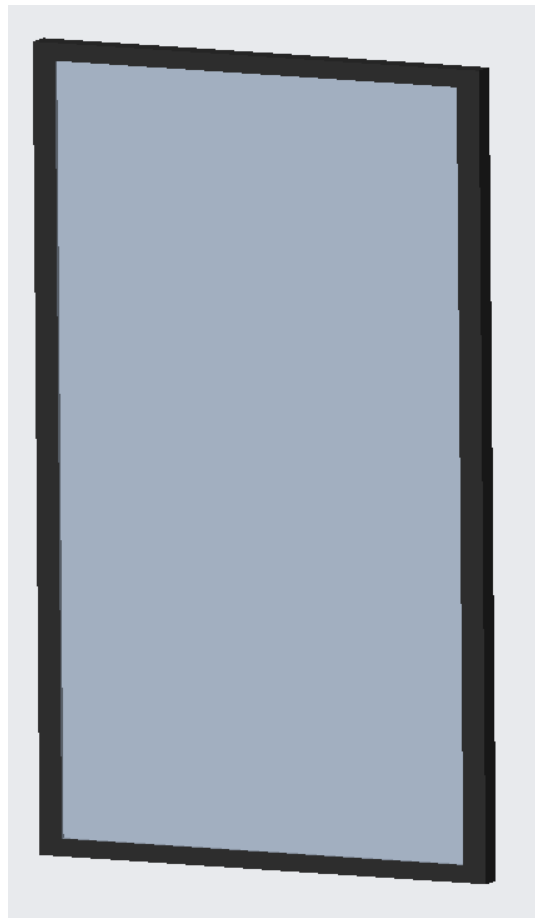
- die Acrylglasscheibe
- der Monitor
- die Lautsprecher
- die Webcam

Diese wurden nach der oben beschriebenen Vorgehensweise annähernd realitätsnah modelliert und sehen als 3D-Modell folgendermaßen aus:

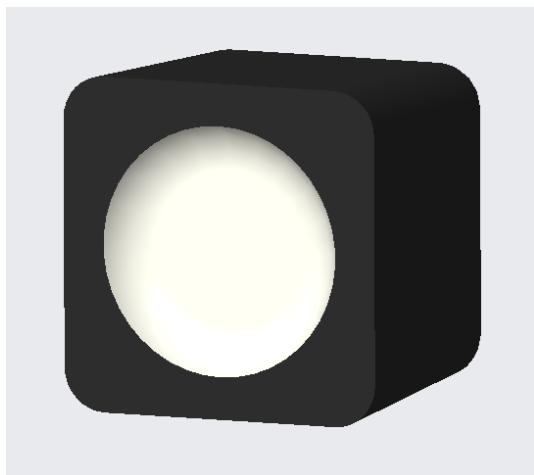
7. Modellierung



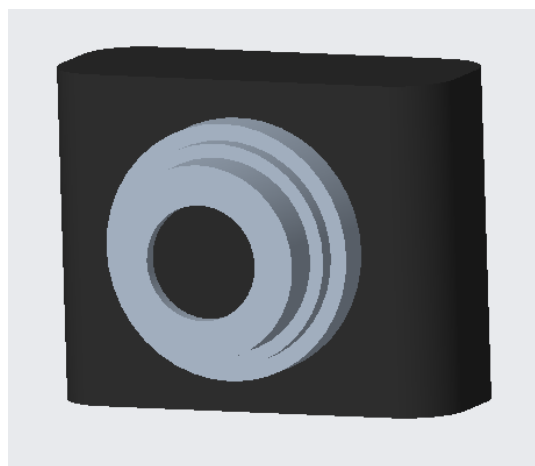
(a) Acrylglas



(b) Monitor



(c) Lautsprecher



(d) Webcam

Abbildung 7.1.: Modelle der bestellten Bauteile

Diese Bauteile bieten die Grundlage für die Modellierung des Holzrahmens, in dem sie letztendlich eingebaut werden.

7.1.2. Vordere Rahmenplatte

Die Vorderseite des Rahmens besteht allgemein aus einer 1cm dicken Holzplatte, welche in der Mitte eine Ausnehmung hat, damit die Spiegelscheibe dahinter erkenntlich wird. Zudem muss diese Platte oben eine Öffnung haben, durch welche die Webcam hindurchsehen kann.

Die Maße der Ausnehmung müssen so gewählt sein, dass die Acrylglasscheibe dahinter auf der Platte aufliegen kann (also kleiner als die Acrylglasscheibe), jedoch groß genug, damit der Monitor dahinter komplett erkenntlich ist (also größer als der Monitor). Gewählt wurden daher die Maße 550mm x 350mm, da das Acrylglas somit auf jeder Seite jeweils 25mm aufliegt und der Monitor dahinter komplett erkennbar ist. Die Ausnehmung an sich ist mittig auf der Holzplatte.

Das Loch für die Webcam muss etwas größer sein als der größte Durchmesser der Webcam. Was mit dem "größten Durchmesser der Webcam" gemeint ist, wird erkenntlich wenn man sich die Webcam genauer ansieht (siehe Abbildung 7.1. (d)). Das Loch muss also so groß sein, dass die gesamte Linse hindurchschaut und die restliche Webcam auf der Kante der vorderen Rahmenplatte aufliegt. Dieser Durchmesser beträgt somit 24mm. Das Loch wird vertikal mittig platziert und hat einen Abstand von der Oberkante von 32,5mm. Dieser Abstand stellt sicher, dass das Loch mittig liegt, sobald die Acrylglasscheibe aufliegt.

Die Grundmaße der Platte (730mm x 530mm x 10mm) stellen sicher, dass die restlichen Elemente des Rahmens auf dieser Platte Platz haben.

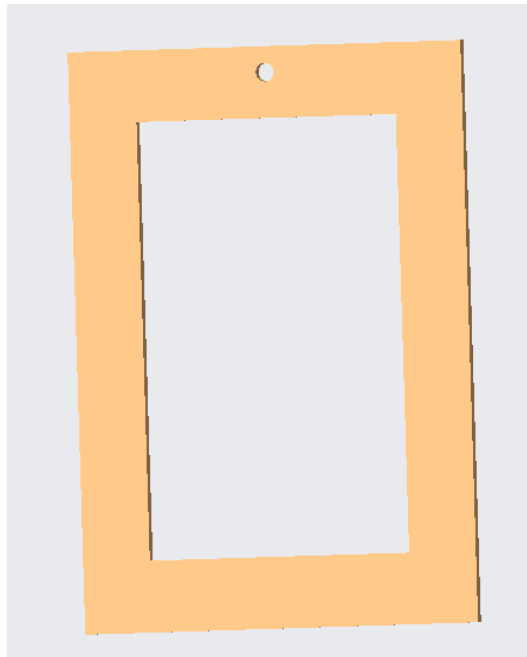


Abbildung 7.2.: Modell der vorderen Rahmenplatte

7.1.3. Oberer Rahmen

Der obere Teil des Rahmens besteht im Grunde aus einem Holzquader, welcher mit seiner Länge die gesamte Breite des Spiegels einnimmt (530mm) und so breit ist wie der Abstand von der Acrylglasscheibe zum oberen Ende der vorderen Rahmenplatte (65mm). Die Höhe von 62mm ist so gewählt, da die untere und die seitlichen Rahmenteile genau dieselbe Höhe haben. Somit kann die hintere Rahmenplatte bündig platziert werden. Warum genau dies eine Höhe von 62mm sein muss, wird dann im Punkt **7.1.5. Seitlicher Rahmen** erkenntlich.

Somit hat der obere Rahmen die Grundmaße 530mm x 65mm x 62mm.

Zu beachten ist hierbei nur eine Ausnehmung, durch die die Webcam hindurchgeführt werden kann. Diese muss auf der Längsseite mittig quer durch das Holzstück gehen und hat die Maße 40mm x 32mm. Da der schwarze Teil der Webcam nur minimal kleinere Maße besitzt, wird diese somit axial stabilisiert und liegt auf der vorderen Rahmenplatte auf. Zudem muss auf der Oberseite dieser Ausnehmung eine kleine Öffnung zur Innenseite des Spiegelrahmens gemacht werden, damit das Kabel der Webcam zum Raspberry Pi geführt werden kann.

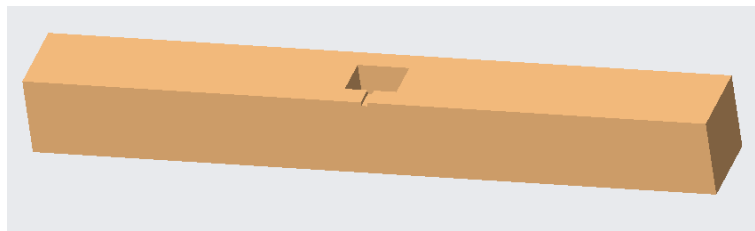


Abbildung 7.3.: Modell des oberen Rahmenteils

Auf dieser Abbildung ist die oben erwähnte Ausnehmung, welche quer durch das Werkstück führt, zu erkennen. Zudem lässt sich eine kleine Öffnung für das Kabel am Ende der Ausnehmung erkennen.

7.1.4. Unterer Rahmen

Dieser Rahmenteil ist im Grunde sehr ähnlich zum oberen Rahmen. Die Grundmaße (530mm x 65mm x 62mm) sind durch dieselben Bedingungen exakt gleich.

Zu beachten ist im Gegensatz beim unteren Rahmen nur, dass wir hier keine Öffnung für die Webcam benötigen. Das Einzige, was hier benötigt wird, ist eine Öffnung für die Stromversorgung. Einerseits muss das Stromkabel des Raspberry Pis und andererseits das des Monitors auf irgend einen Weg nach Außen kommen. Uns schien hierbei die Unterseite des Rahmens als guter Ort.

Da der seitliche Rahmen (wie alle anderen Holzquaderteile, die wir benötigen) eine

7. Modellierung

Breite von 65mm hat, hat die Öffnung für die Stromkabel genau diesen Abstand zum rechten Ende des Werkstücks. Die Öffnung befindet sich also von der Innenseite gesehen genau am rechten Ende des unteren Rahmenteils.



Abbildung 7.4.: Modell des unteren Rahmenteils

7.1.5. Seitlicher Rahmen

Die Breite und die Höhe sind beim seitlichen Rahmen mit 65mm x 62mm genau gleich, wie beim oberen und unteren Rahmenteil. Hier wird auch die Bedeutung der Höhe klar: Die Lautsprecher sollen mittig bei der Seite des Rahmens platziert werden. Somit soll die Rahmenhöhe genau gleich groß sein, wie die Breite der Lautsprecher (62mm).

Da die Lautsprecher jeweils mittig auf der Seite platziert werden sollen, wird die Seite des Rahmens in zwei Teile aufgeteilt und dazwischen der Lautsprecher platziert. Insgesamt muss dieses Rahmenteil somit 275mm lang sein.

Die Grundmaße betragen somit 275mm x 65mm x 62mm.

Damit die Lautsprecher nicht seitlich hinaus fallen können, müssen diese auf irgend eine Art befestigt werden. Unser Einfall war hierbei, dass dieses Rahmenteil am Ende der Längsseite Zähne hat, welche die Lautsprecher einspannen. Da pro Seite jeweils zwei dieser Rahmenteile platziert werden, können somit beide Lautsprecher eingespannt werden.

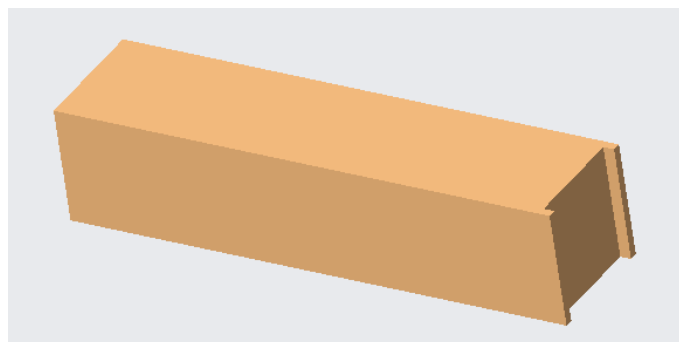


Abbildung 7.5.: Modell des seitlichen Rahmenteils

Insgesamt benötigen wir dieses Bauteil 4 mal, da auf jeder Seite jeweils 2 Stücke platziert werden müssen.

7.1.6. Stabilisierungsplatte für den Monitor

Da die Acrylglasscheibe größer ist, als der aufliegende Monitor und die umschließenden Rahmenteile so groß sind, dass die Acrylglasscheibe genau hinein passt, muss der Monitor axial stabilisiert werden. Deshalb modellierten wir einfach eine Platte, welche mittig genau eine Öffnung mit der Größe des Monitors hat und insgesamt genau gleich groß ist wie die Acrylglasscheibe.

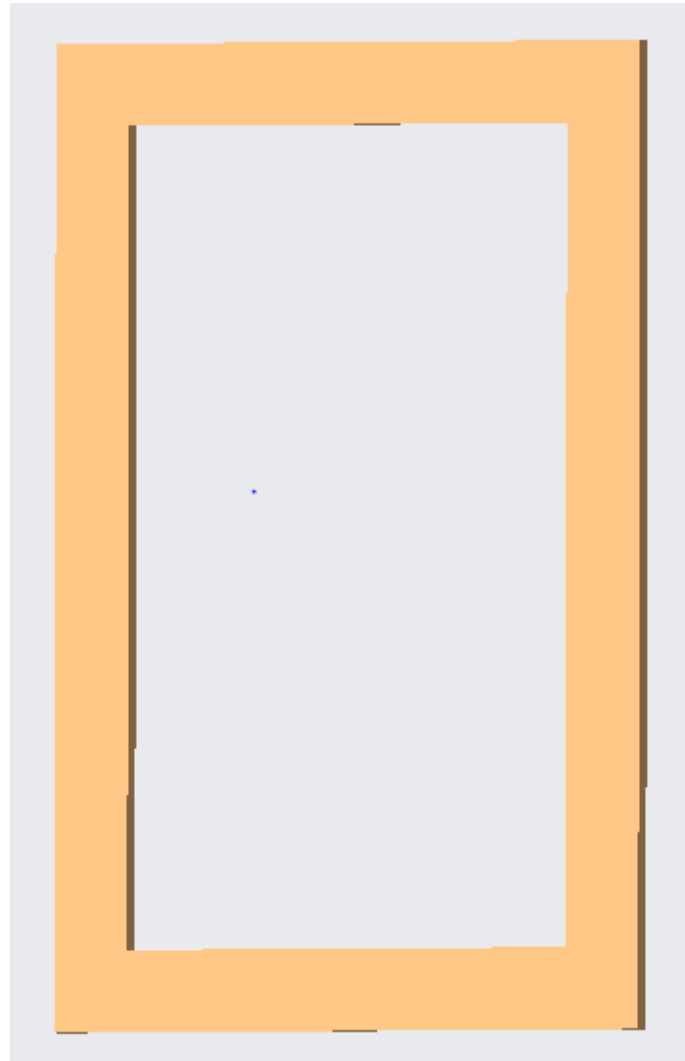


Abbildung 7.6.: Modell der Stabilisierungsplatte für den Monitor

Insgesamt hat diese Platte also die Grundmaße 600mm x 400mm x 10mm (genau gleiche Länge und Breite, wie die Acrylglasscheibe). Die Öffnung in der Mitte beträgt 500mm x 300mm (genau gleich groß, wie der Monitor). Somit ist der Monitor seitlich stabilisiert.

7.1.7. Hintere Stütze für den Monitor

In der Innenseite des Rahmens bildet sich durch unsere Modellierung ein Hohlraum, in dem die gesamte Elektronik platziert wird. Da der Monitor und die Acrylglasscheibe von hinten allerdings durch den Hohlraum nicht gestützt werden, sind diese in diesem Fall noch locker.

Als Lösung für dieses Problem erstellten wir zwei Balken, welche in der Innenseite des Rahmens platziert werden. Ihre Aufgabe ist lediglich den Monitor und die Acrylglasscheibe von hinten zu stützen. Die Grundmaße beziehen sich auf die Innenseite des Rahmens und betragen 400mm x 35mm x 44,5mm. Die Höhe von 44,5mm sorgt dafür, dass diese Balken bündig mit den restlichen Rahmenteilen sind.

Da diese Balken in der internen Verkabelung zu Problemen führen können, haben sie mittig eine Öffnung, welche relativ viel Spielraum für die Kabel bieten.



Abbildung 7.7.: Modell für die Stütze des Monitors

7.1.8. Hintere Rahmenplatte

Abschließend benötigen wir eine Platte, welche die interne Verkabelung von Hinten abdeckt. Die Grundmaße sind genau gleich, wie bei der vorderen Platte (730mm x 530mm x 10mm). Allerdings sind auch bei dieser Platte einige Einzelheiten zu beachten.

Damit der Monitor noch irgendwie angesteuert werden kann, benötigen wir eine Öffnung durch die die Platine des Monitors erreicht werden kann. Diese Platine kann beim Monitor abmontiert werden und an dieser Öffnung befestigt werden. Diese Öffnung hat die Maße 80mm x 9mm.

Letztendlich soll auch die Möglichkeit bestehen, den Rahmen auf der Wand zu montieren. Deshalb haben wir zwei Bohrungen mit jeweils einem Schaft nach oben auf der Rückseite platziert, damit Schrauben diesen Rahmen auf der Wand halten können. Hierbei handelt es sich jeweils um eine 20mm-Bohrung mit einer Schaftbreite von 6mm.

7. Modellierung

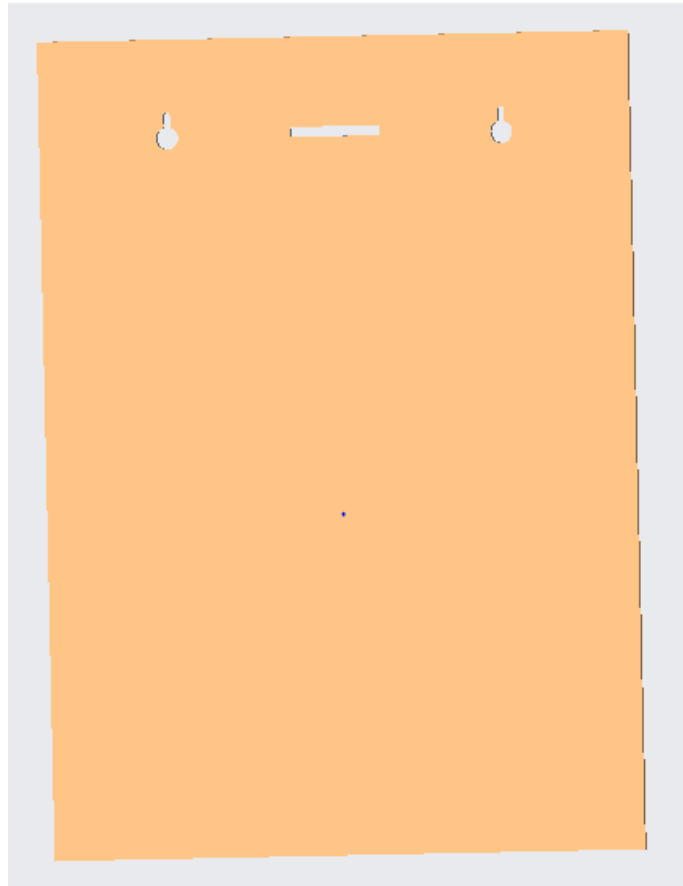


Abbildung 7.8.: Modell der hinteren Rahmenplatte

7.2. Baugruppe (.asm)

Anhand all dieser Einzelteile kann eine Baugruppe erstellt werden, in der das Zusammenspiel all dieser Einzelteile getestet werden kann.

Als Grundlage wird hierbei die vordere Rahmenplatte benutzt, auf welche die Acrylglasscheibe und der Monitor zentral platziert werden. Die Webcam wird so platziert, dass sie durch ihre Öffnung schaut.

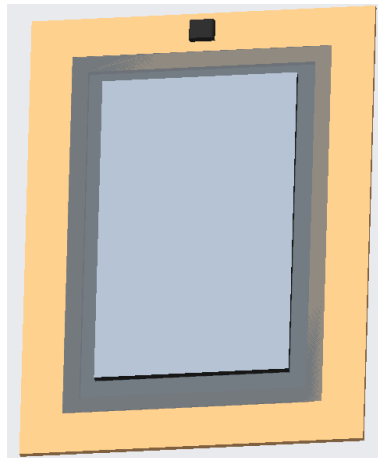


Abbildung 7.9.: Baugruppe Schritt 1

Als nächsten Schritt werden die Stabilisierungsplatte und die umliegenden Rahmenteile (oben, unten und seitlich) hinzugefügt. Hierbei können auch die Lautsprecher eingebaut werden und getestet werden ob die Zähne der seitlichen Rahmenteile diese befestigen.

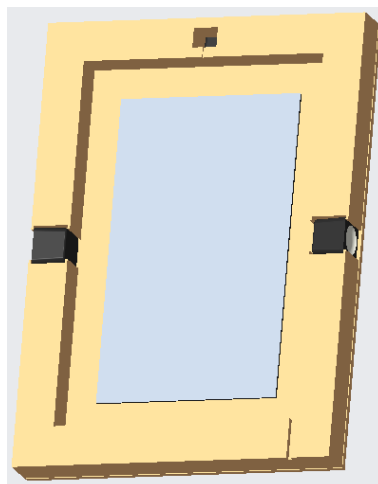


Abbildung 7.10.: Baugruppe Schritt 2

Zu erkennen ist, dass sowohl die Webcam, als auch die Lautsprecher durch diese Teile

7. Modellierung

befestigt werden. Die Acrylglasscheibe und die Stabilisierungsplatte werden durch die seitlichen, oberen und unteren Rahmenteile axial stabilisiert.

Zunächst können die Stützbalken für den Monitor eingebaut werden und getestet werden, ob diese mit den restlichen Teilen bündig sind.

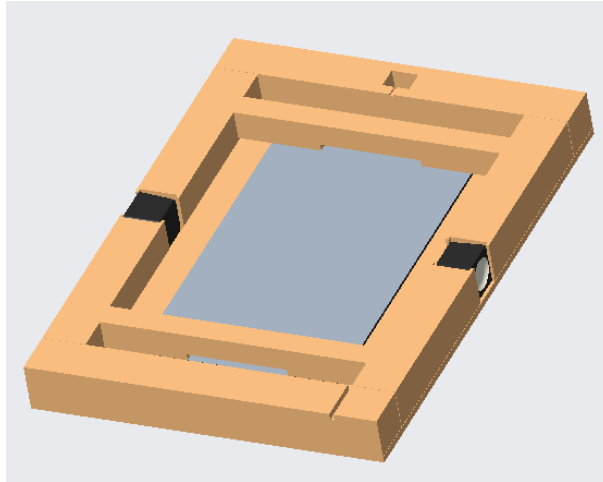


Abbildung 7.11.: Baugruppe Schritt 3

Soweit alles in Ordnung. Als letzter Schritt wird nun die Rückplatte eingebaut.

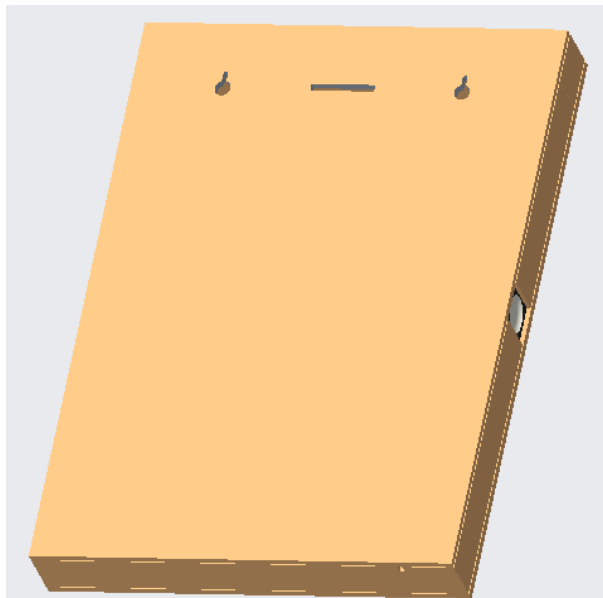
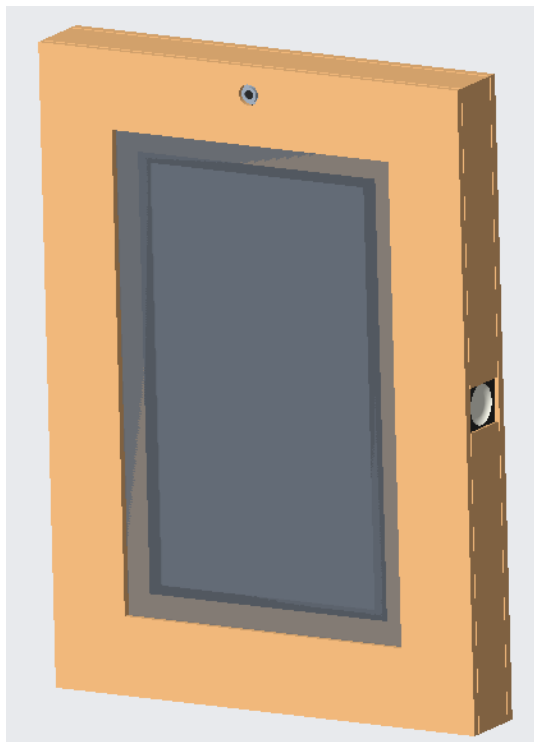


Abbildung 7.12.: Baugruppe Schritt 4

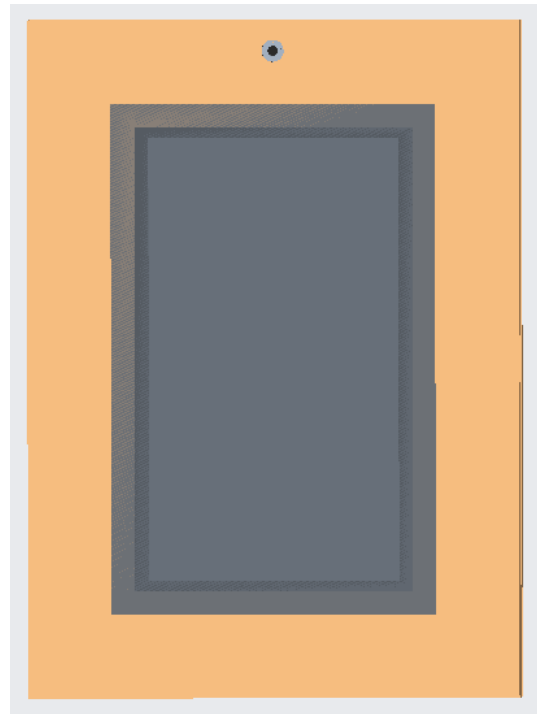
Das Modell des Spiegels steht somit und alle Einzelteile sind in Ordnung. Insgesamt haben wir also einen Spiegelrahmen mit den Maßen 730mm x 530mm x 82mm. Dieser

7. Modellierung

sieht in den wesentlichen Ansichten folgendermaßen aus:



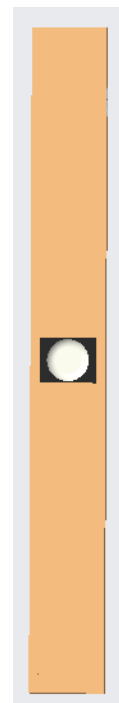
(a) gesamt



(b) vorne



(c) oben



(d) side

Abbildung 7.13.: Baugruppe in allen Ansichten

7.3. Zeichnungsableitungen (.drw)

Da nun klar ist, dass alle modellierten Einzelteile in Ordnung sind, müssen diese im letzten Schritt noch jeweils in eine eigene Fertigungszeichnung abgeleitet werden. Creo kann dies einfach durch die Erzeugung einer .drw-Datei bzw. Zeichnungsableitung machen. In dieser Zeichnungsableitung wird das Bauteil in allen drei Ansichten dargestellt. Die benötigten Maße können dann selbstständig hinzugefügt werden.

Die Zeichnungsableitungen der benötigten Einzelteile sehen folgendermaßen aus:

7.3.1. Vordere Rahmenplatte

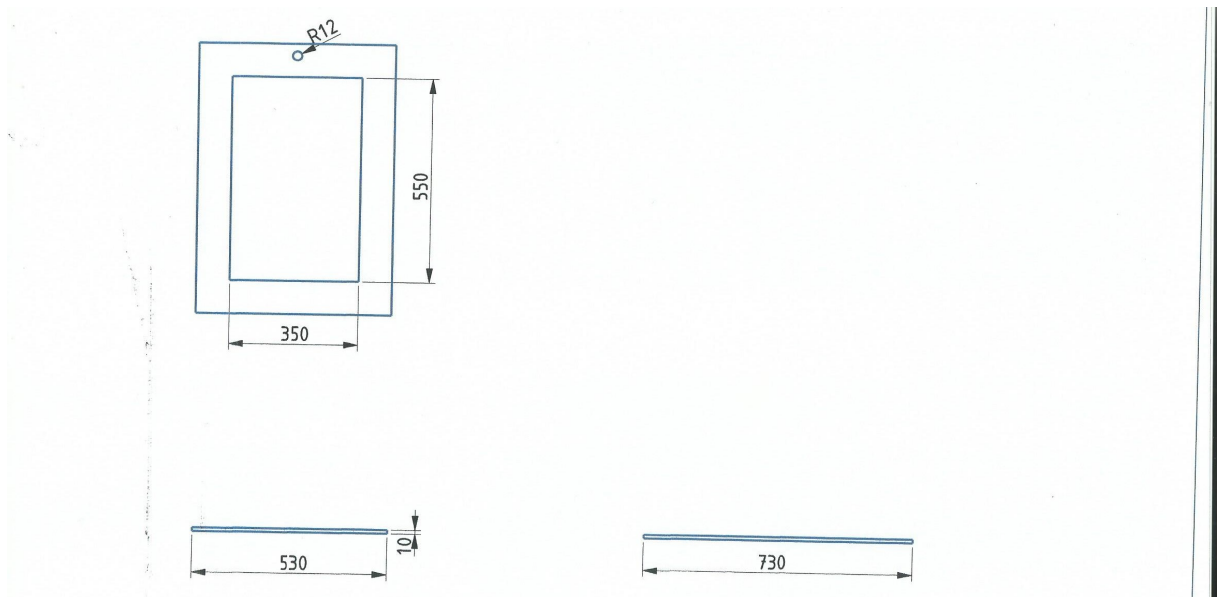


Abbildung 7.14.: Zeichnungsableitung der vorderen Rahmenplatte

7. Modellierung

7.3.2. Oberer Rahmen

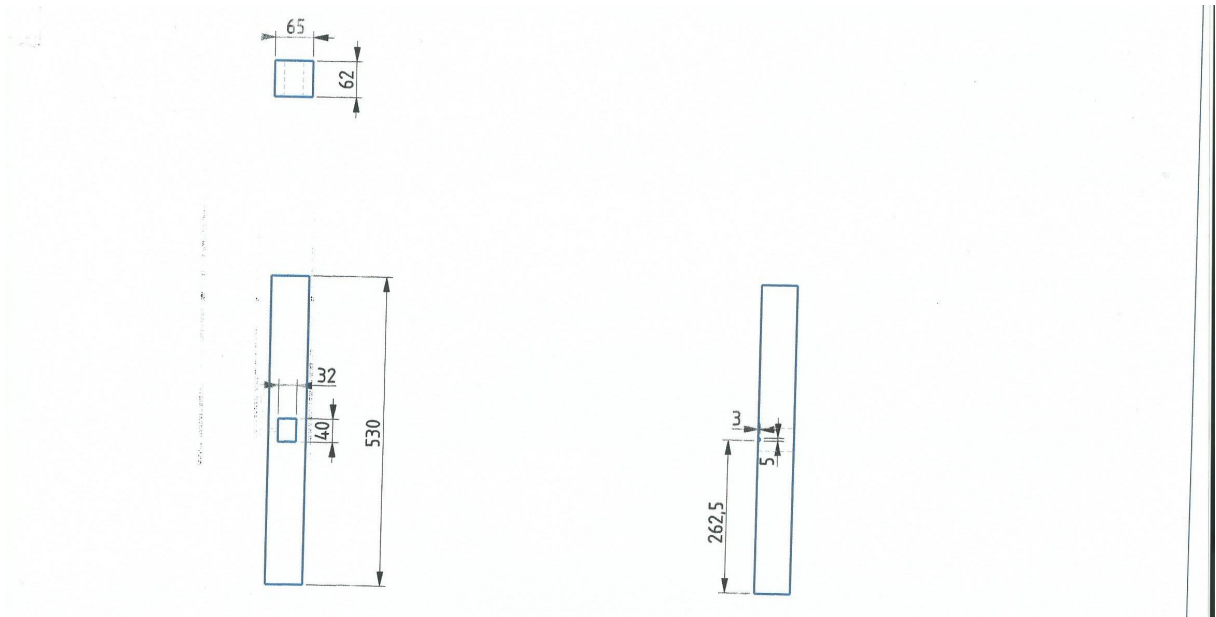


Abbildung 7.15.: Zeichnungsableitung des oberen Rahmenteils

7.3.3. Unterer Rahmen

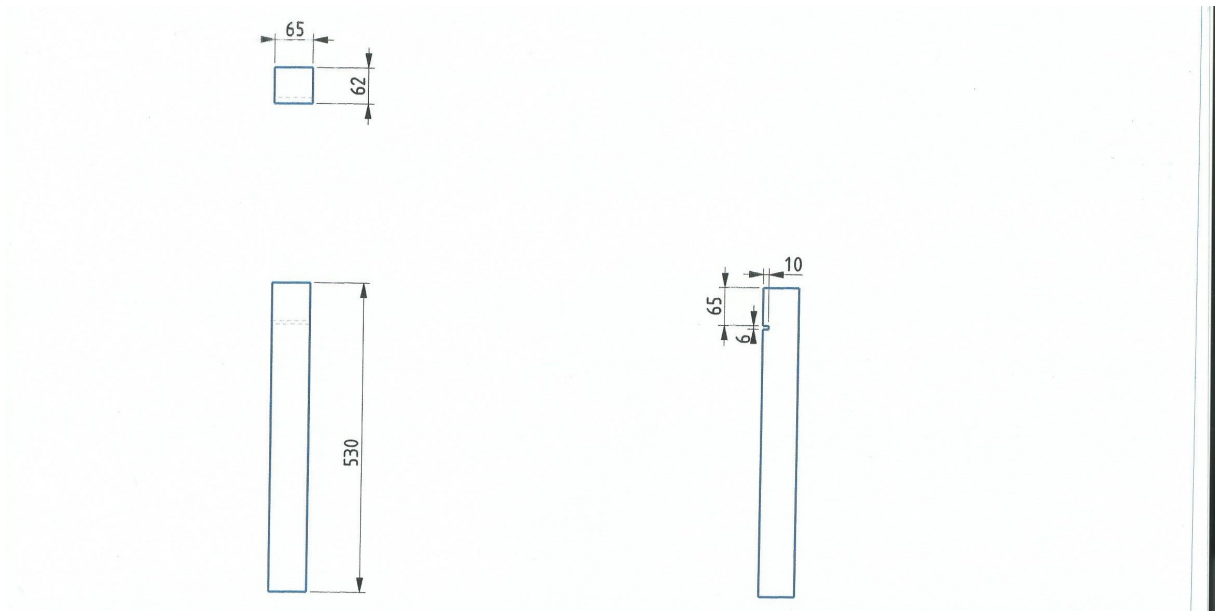


Abbildung 7.16.: Zeichnungsableitung des unteren Rahmenteils

7. Modellierung

7.3.4. Seitlicher Rahmen

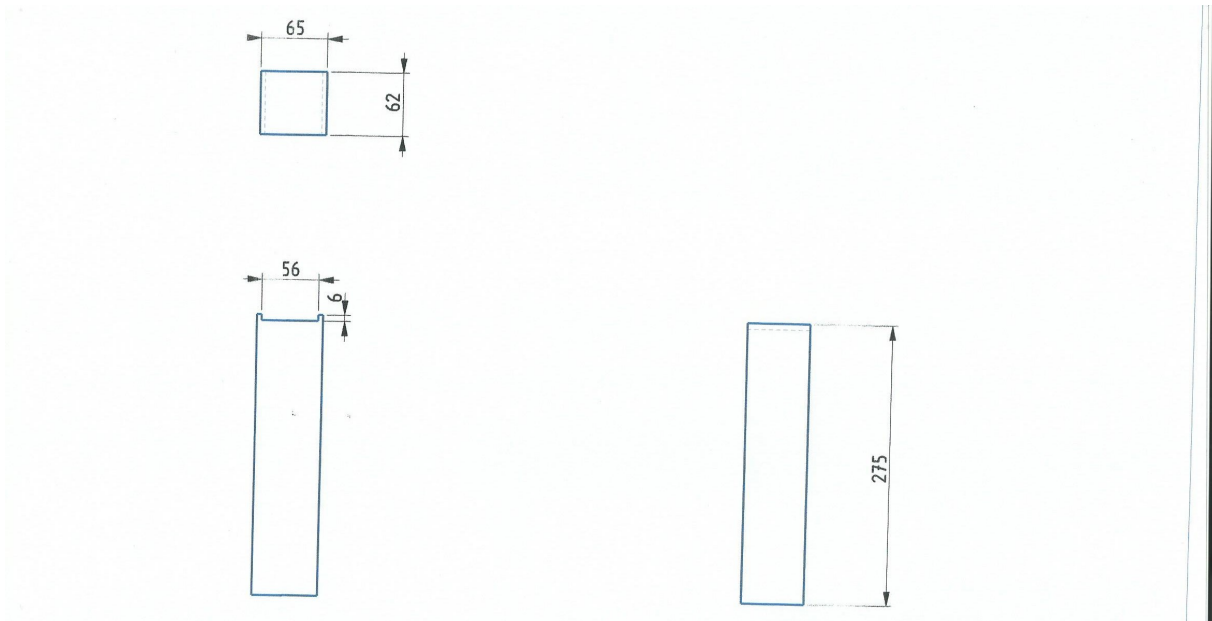


Abbildung 7.17.: Zeichnungsableitung des seitlichen Rahmenteils

7.3.5. Stabilisierungsplatte für den Monitor

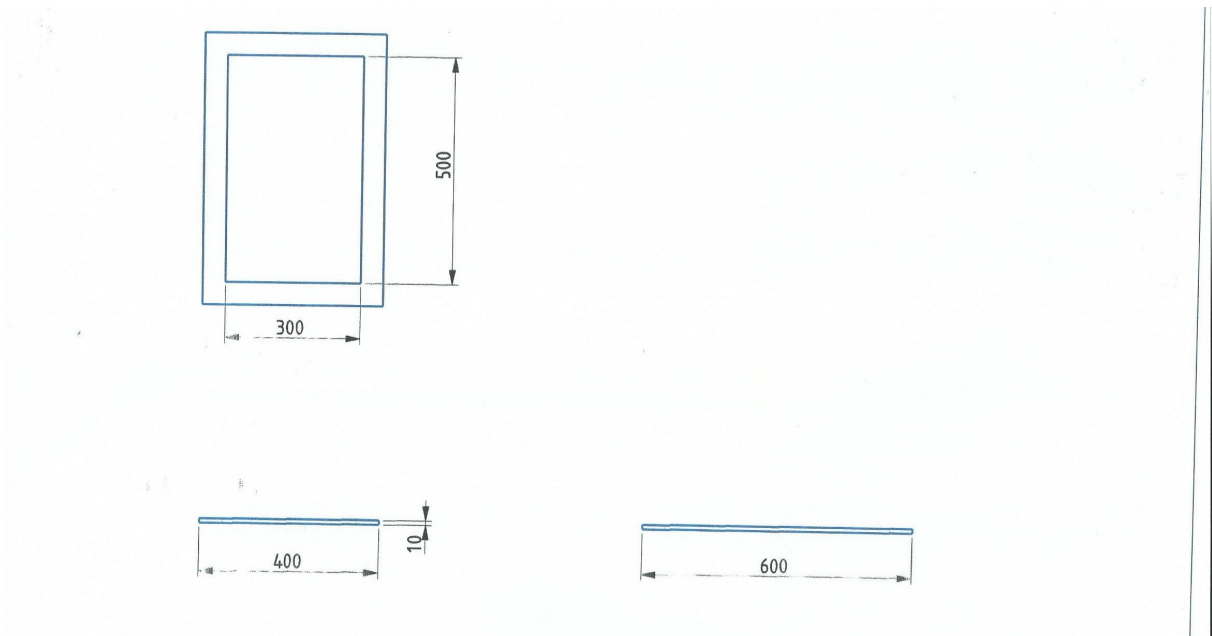


Abbildung 7.18.: Zeichnungsableitung der Stabilisierungsplatte für den Monitor

7. Modellierung

7.3.6. Hintere Stütze für den Monitor

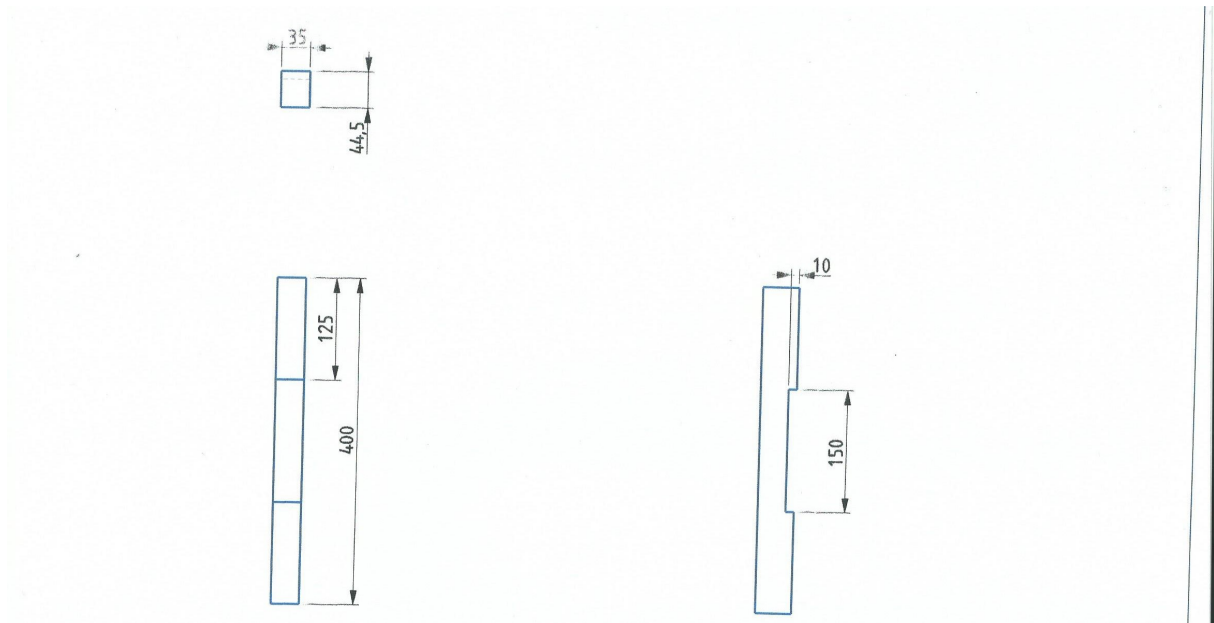


Abbildung 7.19.: Zeichnungsableitung der Stütze des Monitors

7.3.7. Hintere Rahmenplatte

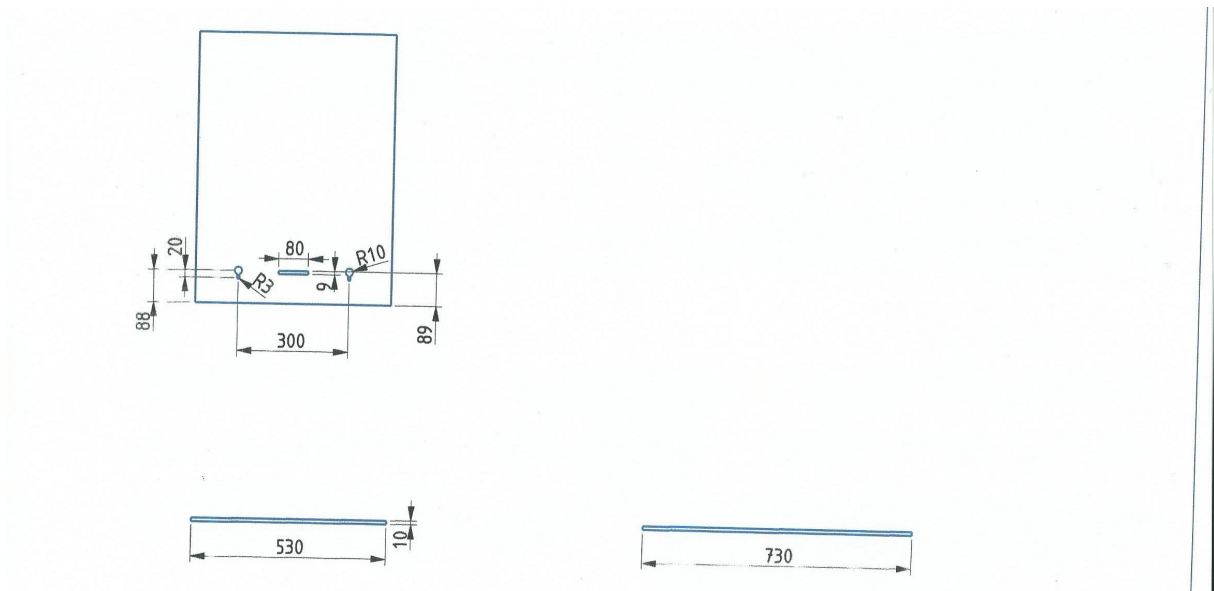


Abbildung 7.20.: Zeichnungsableitung der hinteren Rahmenplatte

8. Fertigung

Wichtig für die Fertigung der benötigten Einzelteile sind im Grunde nur die oben dargestellten Zeichnungsableitungen. Mit diesen können die Einzelteile in einer Holzwerkstatt gefertigt werden.

In den folgenden Absätzen wird der Fertigungsprozess für jedes Einzelteil beschrieben. Anschließend werden noch abschließende Schritte (z.B.: Zusammenbau, Lackierung, etc.) beschrieben.

8.1. Vordere Rahmenplatte

Da sowohl die vordere als auch die hintere Platte beim Kauf bei OBI direkt auf die passenden Grundmaße geschnitten wurden (730mm x 530mm) ersparten wir uns diesen Schritt. Zu beachten ist hierbei nur, dass die gekauften Platten eine Dicke von 8mm statt 10mm besitzt. Dies bewirkt in unserem Fall jedoch nur einen positiven Effekt: Der Spiegel wird insgesamt ein wenig leichter.

Bei der Bearbeitung der Vorderplatte mussten 2 Bearbeitungsschritte gemacht werden:

1. die Ausnehmung für die Acrylglasscheibe
2. das Loch für die Webcam

Angefangen wurde hierbei mit der Ausnehmung für die Acrylglasscheibe. Hierbei markierten wir diese zuerst laut Zeichnung und schnitten diese dann mit der Handkreissäge. Dieser Schritt wurde mit einer Schiene gemacht, welche sicherstellt, dass die Handkreissäge in einer geraden Linie schneidet.

Die Handkreissäge alleine reicht mit dieser Methode allerdings nicht aus, da hier an den Enden der einzelnen Schnitte noch kleine Überreste bleiben. Dies ist die Ursache der runden Form des Sägeblatts. Um die Schnitte zu vollenden wurden diese Überreste noch mit der Japansäge entfernt.

8. Fertigung



Abbildung 8.1.: Vollendung des Schnitts durch Japansäge

Bei der Bohrung gab es nicht viel zu beachten. Wir markierten den Punkt und bohrten hier mit einem Akkubohrer. Dabei verwendeten wir einen Bohraufsatz mit Durchmesser 24mm.



Abbildung 8.2.: Markierung der Bohrung für die Webcam

Anschließend wurden noch alle Seiten der Platte mit der Schleifmaschine nachgeschliffen.

8.2. Oberer Rahmen

Die benötigte Breite und Höhe des oberen, unteren und der seitlichen Rahmenteile betragen allesamt 65mm x 62mm. Deshalb wurden alle Rohteile für diese Seiten (90mm x 90mm x 1000mm) mit der Bandsäge auf unsere gewünschte Größe zugeschnitten. Die Grundmaße dieser Teile unterscheiden sich somit nur in ihrer Länge.

Beim oberen Rahmenteil beträgt diese Länge 530mm. Diese wurde durch die Kappsäge erreicht. Da dieses Werkstück nun auf seine Grundmaße geschnitten ist, können die speziellen Bearbeitungen realisiert werden. Hierbei handelt es sich primär um die rechteckige Ausnehmung (40mm x 32mm) quer durchs Werkstück für die Webcam.

Dazu verwendeten wir die Oberfräse. Wir markierten dieses Rechteck auf beiden Seiten und schnitten mit der Oberfräse auf beiden Seiten solange hinein, bis das Werkstück durchdrungen war.



Abbildung 8.3.: Schnitt des Webcamlochs mit Oberfräse

Da mit der Oberfräse die rechteckige Form nicht erreicht werden konnte, schnitten wir die Überreste mit dem Schnitzmesser weg. Wir testeten, ob die Webcam hindurch passt. Sobald dies der Fall war, war der Schnitt vollendet.

Die Ausnehmung für das Kabel wurde ganz einfach mit der Oberfräse realisiert. Dazu verwendeten wir einen 5mm-Fräser und schnitten 3mm in das Werkstück hinein.

8. Fertigung

Für die Verbindung zu den seitlichen Rahmenteilen, wurden bei der Fläche, welche zur Innenseite des Rahmens zeigt, zwei 8mm-Bohrungen gemacht. Diese sind vertikal gesehen mittig und haben an den Enden jeweils einen Abstand von 32,5mm. Da in diese Löcher 40mm lange Dübel befestigt werden, muss die Bohrung 20mm ins Werkstück gehen.

Anschließend wurde das gesamte Teil noch geschliffen.

8.3. Unterer Rahmen

Der untere Rahmen wurde, wie der obere Rahmen, mit der Kappsäge auf eine Länge von 530mm geschnitten.

Wie schon bei der Modellierung beschrieben, musste bei diesem Teil fast nichts bearbeitet werden. Das Einzige, was geschnitten werden musste, war die Ausnehmung für die Stromkabel des Raspberry Pis und des Monitors. Hierbei wurde diese wieder markiert und mit der Oberfräse geschnitten. Dazu verwendeten wir einen 6mm-Fräser, welcher 10mm in das Werkstück hineinschnitt. Dieser Schnitt geht einmal quer durch das Werkstück.

Wie auch bei dem oberen Rahmen musste hier für die Dübel-Verbindung zum seitlichen Rahmen jeweils eine 20mm tiefe Bohrung mit einem Abstand von 32,5mm an den Enden gemacht werden (siehe 8.2. Oberer Rahmen).

Wie jedes andere Werkstück wurde auch dieses Teil einmal komplett geschliffen.

8.4. Seitlicher Rahmen

Die Grundmaße betragen 275mm x 65mm x 62mm. Da die Breite und die Höhe zuvor schon mit der Bandsäge zugeschnitten wurden, musste hier nur noch mit der Kappsäge die Länge von 275mm erreicht werden.

Anschließend wurde an einem Ende mit der Oberfräse eine Ausnehmung mit der Breite von 56mm quer durch das Werkstück realisiert. Diese Ausnehmung ist 6mm tief.

8. Fertigung

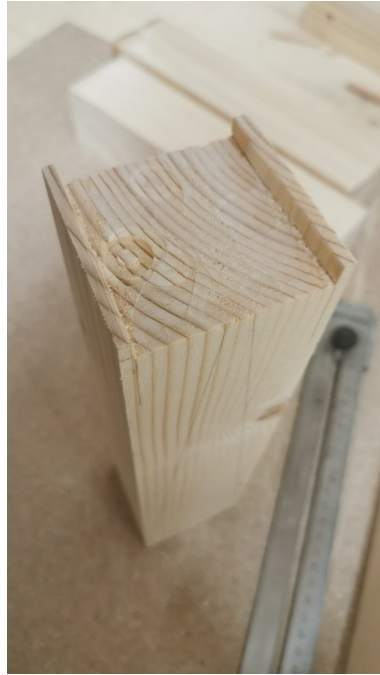


Abbildung 8.4.: Ausnehmung zum Einspannen der Lautsprecher

An der gegenüberliegenden Seite wurde wieder eine 20mm tiefe Bohrung für die Dübel-Verbindung gemacht. In diesem Fall ist sie direkt mittig.



(a) Markieren der Bohrung



(b) Bohren

Abbildung 8.5.: Bohrung für die Dübel-Verbindung

8. Fertigung

Diese Bohrung wurde auch auf der Innenseite des Werkstücks gemacht, um die Dübel-Verbindung zur Monitor-Stütze zu ermöglichen.

Daraufhin erfolgte wieder das Schleifen des Werkstücks. Insgesamt musste dieses Teil 4 mal gefertigt werden, da für jede Seite jeweils 2 Teile benötigt werden, um die Lautsprecher einzuspannen.

8.5. Stabilisierungsplatte für den Monitor

Die Stabilisierungsplatte musste im Gegensatz zu den anderen Platten erst auf ihre Grundmaße geschnitten werden. Verwendet wurde eine Platte mit den Dimensionen 700mm x 450mm x 10mm. Mit der Bandsäge brachten wir sie auf die Maße 600mm x 400mm x 10mm.

Bei der weiteren Bearbeitung benötigten wir nur eine zentrale Ausnehmung von 500mm x 300mm für den Monitor. Diese wurde genau gleich wie die Ausnehmung bei der vorderen Platte gemacht (zuerst mit der Handkreissäge grundlegend und dann mit der Japansäge). Sobald der Rahmen des Monitors in dieses Rechteck passt, ist der Schnitt vollendet.



Abbildung 8.6.: Vollendeter Schnitt bei der Stabilisierungsplatte

Anschließend musste diese Platte wieder mit der Schleifmaschine bearbeitet werden.

8.6. Hintere Stütze für den Monitor

Die Grundmaße dieses Werkstück (400mm x 35mm x 44,5mm) wurden wieder mit der Bandsäge erreicht. Auf der Unterseite wurde noch eine Ausnehmung von 150mm Länge und 10mm Tiefe quer durch das Werkstück gemacht. Dafür wurde wieder die Oberfräse benutzt und die Überreste mit dem Schnitzmesser entfernt.



Abbildung 8.7.: Ausnehmung bei der Stütze für den Monitor

An beiden Enden dieser Stütze musste wieder eine Bohrung für die Dübel-Verbindung gemacht werden. Dabei muss die Position so gewählt werden, dass die Stütze bündig zu den umliegenden Teilen ist. Danach wird dieses Teil geschliffen. Insgesamt benötigen wir wieder 2 Teile dieses Werkstücks, weshalb dieser gesamte Prozess auch 2 mal durchgeführt werden muss.

8.7. Hintere Rahmenplatte

Die hintere Rahmenplatte wurde beim Kauf, genau wie die vordere, direkt auf Maß zugeschnitten. Somit ersparten wir uns diesen Schritt. Insgesamt mussten hier wieder 2 Bearbeitungsschritte durchgeführt werden:

1. rechteckige Ausnehmung für die Platine des Monitors
2. 2 Bohrungen zum Aufhängen des Spiegelrahmens

Die rechteckige Ausnehmung markierten wir laut Zeichnung auf der Rückseite und frästen diese dann mit der Oberfräse weg. Jedoch fiel uns dann auf, dass die Knöpfe der Platine durch die Dicke der Platte fast nicht zu erreichen waren. Deshalb markierten wir uns die Form der Platine auf der Rückseite und frästen 4mm tief hinein. Am Schluss wurde die Platine dann mit einer Heißkleber-Pistole in diese Form hinein geklebt.



(a) Markieren der Ausnehmung



(b) Kleben der Platine

Abbildung 8.8.: Befestigung der Platine

Für die Bohrungen zum Aufhängen des Spiegelrahmens wurden die Markierungen wieder laut Zeichnung gesetzt. Somit bohrten wir jeweils mit einem 20mm-Bohrer und danach mit einem 6mm-Bohrer (20mm über der vorherigen Bohrung). Diese zwei Bohrungen wurden dann mit der Oberfräse (6mm-Aufsatz) miteinander verbunden.

Als nächsten Schritt mussten wir uns noch überlegen, wie die Webcam von hinten gestützt wird, ohne beschädigt zu werden. Hierfür verwendeten wir einen kleinen Holzklötz, welcher in die Ausnehmung beim oberen Rahmenteil (40mm x 32mm) hinein passt. Auf diesem Holzklötz wurde ein Schwamm geklebt. Dieser stützt letztendlich die Webcam, ohne sie zu beschädigen.

8. Fertigung



Abbildung 8.9.: Stütze für die Webcam

Dieser Klotz wurde dann auf der richtigen Position auf der Rückseite der Hinterplatte durch eine Schraube befestigt. Anschließend erfolgt wieder ein Schleifprozess für die gesamte Platte.

8.8. Abschließende Schritte

8.8.1. Befestigung der umliegenden Rahmenteile

Bei der Befestigung der umliegenden Rahmenteile (vorne bzw. hinten, seitlich und Stütze für den Monitor) entschieden wir uns für eine Dübel-Verbindung. Dafür waren auch die oben genannten Bohrungen bei diesen Teilen.

Wir benutzen hierbei Dübel mit einem Durchmesser von 8mm und einer Länge von 40mm. Somit musste auf jeder Seite eine 20mm tiefe Bohrung gemacht werden, welche einen Durchmesser von 8mm hat. Anschließend wurde der Dübel mit Hilfe von Holzleim in beide Seiten hinein geleimt. Der Leim wurde auch auf die aufliegenden Flächen aufgetragen, damit auch eine stabile Verbindung besteht.



(a) Dübel mit Bohrung



(b) Leimen der Verbindung

Abbildung 8.10.: Dübel-Verbindung

8.8.2. Befestigung der vorderen und hinteren Platte

Bei der Befestigung der Platten wurden rund um die Platte 4x50mm Torx-Schrauben verwendet. Dafür mussten diese Bohrungen erst markiert, danach zentriert und letztendlich gebohrt werden. Die Positionen der Bohrungen wurde so gewählt, dass die Torx-Schraube in eine der umliegenden Rahmenteile (vorne, hinten, seitlich und Stütze für den Monitor) gebohrt wird.

8. Fertigung



(a) Markieren der Bohrungen



(b) Ansetzen der Torx-Schrauben

Abbildung 8.11.: Platten-Verbindung

8.8.3. Grundlegender interner Aufbau

Nachdem alle Teile befestigt werden können, kann ein grundlegender interner Aufbau gemacht werden. Sobald die vordere Platte mit allen umliegenden Teilen durch Torx-Schrauben befestigt wurde (davor müssen die Acrylglasscheibe, der Monitor und die Stabilisierungsplatte hineingelegt werden), kann die gesamte Hardware platziert werden.



Abbildung 8.12.: Interner Aufbau

8. Fertigung

Links und rechts können hier die Lautsprecher erkannt werden. Diese passen direkt in ihre Plätze und können durch die Zähne der seitlichen Rahmenteile auch nicht verrutschen. Die Webcam wird nach oben zum oberen Rahmenteil geführt. All diese Bauteile sind mit dem Raspberry Pi verbunden (der Audioport der Lautsprecher auch noch mit dem Monitor). Der Monitor und der Raspberry Pi sind zusätzlich noch mit einem HDMI-zu-MicroHDMI-Kabel verbunden. Die Stromkabel für den Monitor und den Raspberry Pi werden unten rechts hinaus geführt. Im Raspberry Pi steckt zudem noch der Wireless-USB-Plug für die Tastatur.

Das einzige Problem hier ist noch die Befestigung des Raspberry Pis. Hierbei haben wir einfach vier kleine Holzstücke auf dem Rahmen des Raspberry Pis geklebt und diese vier Holzteile dann auf den Monitor.



Abbildung 8.13.: Befestigung des Raspberry Pis

Die vier Holzstücke sorgen dafür, dass es einen Abstand zu dem Lüfter des Raspberry Pis und dem Monitor gibt. Ansonsten würde der Raspberry Pi leicht überhitzen. Dadurch dass sich diese Holzstücke auf der oberen Seite des Rahmens befinden, wird nur dieser permanent auf den Monitor geklebt. Der Raspberry Pi kann somit für Wartungszwecke leicht abmontiert werden.

8.8.4. Lackierung des Rahmens

Schlussendlich wird der Spiegelrahmen noch als letzten Schritt von außen lackiert. Wir haben uns für schwarz entschieden, da der Rahmen somit ein edleres Aussehen benutzt. Wir erwarben eine schwarze Sprühdose bei OBI und lagen den Spiegelrahmen ohne die Elektronik auf den Tisch. Daraufhin sprühten wir einfach alle erkenntlichen Flächen mit der Dose an und warteten einige Stunden bis die Farbe trocknete.

8. Fertigung



Abbildung 8.14.: Sprühen des Spiegelrahmens

Der fertig gesprühte Spiegelrahmen sieht wie folgt aus:



Abbildung 8.15.: Fertig besprühter Spiegelrahmen

Nachdem wir den fertig besprühten Spiegelrahmen hatten, mussten wir nur noch die Hardware einbauen und wir besaßen einen völlig fertig gebauten Smart-Mirror.

Teil IV.

Backendstruktur

9. Datenbank

9.1. Datenbankmodell

Der wahrscheinlich wichtigste Schritt für eine erfolgreiche Backendstruktur ist die Modellierung eines effizienten Datenbankmodells. Hierbei muss man grundlegende Überlegungen über das Programm treffen. Welche Daten werden benötigt? Welche Daten hängen zusammen? Wie hängen diese Daten zusammen? Zum Beantworten dieser und weiteren Fragen bietet es sich an, ein relationales Datenbankschema zu erstellen, in dem alle notwendigen Tabellen mit den dazugehörigen Parametern veranschaulicht werden:

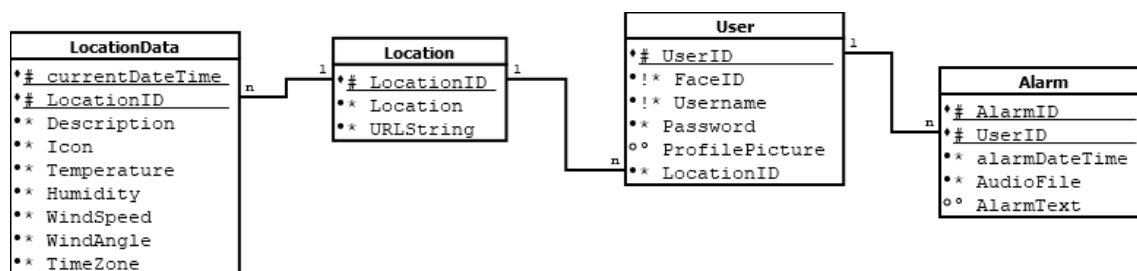


Abbildung 9.1.: Datenbankschema

In dieser Darstellung sind einige Symbole zu beachten:

- **Kopfzeile:** In der Kopfzeile einer Tabelle ist der Name der Tabelle angegeben. Über diesen Namen wird die Tabelle in der Datenbank gespeichert.
- **Werte:** Unter der Kopfzeile stehen die einzelnen Werte der Tabelle. Jeder Datensatz der Tabelle muss aus genau diesen Werten bestehen. Vor diesen Werten stehen Symbole, welche genauere Informationen über diesen Wert darstellen.
- **#:** Das #-Symbol definiert ein Schlüsselattribut. Dieses wird zur einfachen Erkennung nochmals unterstrichen. Das Schlüsselattribut unterscheidet jeden einzelnen Datensatz dieser Tabelle voneinander und kann auch aus mehreren Werten gleichzeitig bestehen. Jeder Datensatz der Tabelle besitzt ein anderes Schlüsselattribut.
- *****: Das *-Symbol bedeutet, dass dieser Datenwert nicht null (undefiniert) sein darf. Falls dies der Fall ist darf dieser Wert bei der Erzeugung eines neuen Eintrags nicht leer sein.
- **!:** Mit dem !-Symbol wird dargestellt, dass es sich bei diesem Wert um einen eindeutigen Wert handelt (auch wenn dieser kein Schlüsselattribut ist). Ähnlich

wie beim Schlüsselattribut unterscheidet sich jeder Datensatz durch die Werte, welche mit ! markiert sind.

- °: ° beschreibt das Gegenstück zu * (nicht nullierbar). Werte, die mit ° markiert sind, können also bei Erzeugung eines neuen Eintrags undefiniert bleiben (null). Man nennt solche Werte auch **nullierbar**.
- **Beziehungen:** Beziehungen zwischen Tabellen werden durch Linien gekennzeichnet. Hierbei werden zwei Werte, welche den gleichen Wert besitzen, miteinander verbunden. Werte von anderen Tabellen, welche zur Verbindung benötigt werden, werden auch **Foreign Keys** genannt. In wie Fern diese Werte in Zusammenhang stehen, wird über **1** (nur ein Datensatz kann mit dieser Tabelle in Verbindung stehen z.B.: ein User kann nur einen Standort als Wunschstandort angeben) oder **n** (mehrere Datensätze können mit dieser Tabelle in Verbindung stehen z.B.: mehrerer User können den selben Standort als Wunschstandort angeben) angegeben.

Sieht man sich nun unser konkretes Datenbankschema genauer an (siehe Abbildung 9.1), dann erkennt man, dass vier Tabellen benötigt werden:

- **Location:** In der Location-Tabelle werden grundlegende Informationen über die Standorte der API gespeichert. Diese werden zum Programmstart aus einer lokalen Datei ausgelesen (city.list.json) und in die Datenbank eingetragen. Diese Informationen sind eine eindeutige locationID, der Name des Standorts und die URL, über die der API-Call für die Wetterdaten gemacht werden kann.
- **User:** Bei der User-Tabelle werden alle nutzer-relevanten Informationen gespeichert. Hierbei handelt es sich um eine eindeutige ID, dem Dateipfad zu den gespeicherten Gesichtswerten (faceID), einem Usernamen, einem Passwort, einem Profilbild und um einen Verweis auf den gewünschten Standort für die Wetterdaten.
- **LocationData:** Die LocationData-Tabelle speichert zeitlich abhängige Einträge für die Standorte. Diese Werte sollen stündlich aktualisiert bzw. ein neuer Eintrag pro Standort hinzugefügt werden. Diese Datensätze werden daher über den Zeitpunkt und über einen Verweis auf den Standort definiert. Alle anderen Daten sind Werte über das aktuelle Wetter.
- **Alarm:** Jeder Nutzer kann über diese Tabelle bestimmte Meldungen abspeichern, welche an bestimmten Zeitpunkten mit einer Audio-Datei begleitet abgespielt werden (z.B.: Wecker). Diese werden anhand einer eindeutigen ID und des jeweiligen Users genau definiert. Zudem muss ein genauer Zeitpunkt und der Dateipfad zur gewünschten Audio-Datei angegeben werden. Zusätzlich kann der Nutzer auch einen Text angeben, welcher beim Alarm erscheint.

9.2. SQL-Code

Anhand des oben dargestellten Datenbankmodells kann der SQL-Code für die Erzeugung der Datenbankstruktur erstellt werden:

```
1 drop table if exists location;
2 drop table if exists user;
3 drop table if exists locationData;
4 drop table if exists alarm;
```

Als ersten Schritt werden alle oben angegebenen Tabellen gelöscht, sollten diese existieren. Dies gewährleistet, dass die Datenbank zu Beginn leer ist und die Erzeugung der einzelnen Tabellen erfolgreich abläuft.

```
1 create table location (
2 locationID int primary key,
3 location varchar(100) not null,
4 urlString varchar(255) not null
5 );
```

Die erste Tabelle, welche erzeugt wird, ist die einzige, die keine Fremdschlüssel (Foreign Keys) beinhaltet. Würde sie einen Fremdschlüssel besitzen, müsste auf diesen verwiesen werden. Dies kann nicht funktionieren, da noch keine Tabellen existieren.

Hier werden einfach die drei oben angezeigten Attribute mit ihrem Datentyp definiert. Werte, die mit einem * gekennzeichnet sind, erhalten hier das **not null**-Keyword. Das Schlüsselattribut muss mit **primary key** definiert werden.

```
1 create table user (
2 _id integer primary key,
3 faceID varchar(255) not null unique,
4 username varchar(50) not null unique,
5 password varchar(50) not null,
6 profilePicture varchar(255),
7 locationID int not null,
8 foreign key(locationID) references location(locationID)
9 );
```

Bei der User-Tabelle wird das `_id` Attribut zum ersten Mal verwendet. Das Attribut `_id` ist ein bereits vorgegebenes Attribut von SQLite, welches bei jedem neuen Datensatzeintrag inkrementiert (um 1 erhöht) wird. Dieses Attribut hat also genau die gleiche Aussage, wie die `userID`, deswegen wird dieses hier auch genau so verwendet.

Zudem lässt sich hierbei das **unique**-Keyword erkennen. Es sagt aus, dass es sich bei diesem Attribut um einen eindeutigen Wert handelt. Dieses muss folglich bei allen mit ! gekennzeichneten Attributen hinzugefügt werden.

Auch der Fremdschlüssel muss als solcher definiert werden und auf das entsprechende Attribut der referenzierten Tabelle hinweisen (**foreign key**).

```
1 create table locationData (
2 currentDateTime timestamp default current_timestamp,
3 locationID int,
```


9. Datenbank

```
4 | description varchar(100) not null,  
5 | icon varchar(255) not null,  
6 | temperature double not null,  
7 | humidity double not null,  
8 | windSpeed double not null,  
9 | windAngle double not null,  
10 | timezone int not null,  
11 | primary key(currentDateTime, locationID),  
12 | foreign key(locationID) references location(locationID)  
13 | );
```

Die LocationData-Tabelle besitzt einen Timestamp. Dieser wird standardmäßig auf die aktuelle Uhrzeit gestellt (**default current_timestamp**). Das bedeutet, dass bei jedem Eintrag, bei dem die currentDateTime nicht explizit angegeben wird, dieser Wert auf den Eintragszeitpunkt gesetzt wird.

Hier gibt es auch einen zusammengesetzten Schlüssel, weshalb dieser in einer eigenen Zeile definiert werden muss, indem man alle Schlüsselattribute zusammenfasst. Auf den Fremdschlüssel locationID muss wieder referenziert werden.

```
1 | create table alarm (  
2 | _id integer,  
3 | userID int,  
4 | alarmDateTime timestamp not null,  
5 | audioFile varchar(255) not null,  
6 | alarmText varchar(255),  
7 | primary key(_id, userID),  
8 | foreign key(userID) references user(_id)  
9 | );
```

Auch bei der Alarm-Tabelle wird für die alarmID wieder das **_id**-Attribut verwendet. Der Schlüssel setzt sich aus den Attributen **_id** und **userID** zusammen. Hier muss wieder auf den Fremdschlüssel **userID** referenziert werden.

Anhand diesem SQL-Codes kann eine stabile Backendstruktur realisiert werden.

10. Implementierte Klassen

10.1. package database

10.1.1. class DBManager

Bei der Klasse DBManager handelt es sich um eine Wrapper-Klasse mit der bestimmte datenbankrelevante Funktionen einfach aufgerufen werden können. Der Benutzer einer Instanz der Klasse DBManager kann durch dieses Objekt die Datenbank bearbeiten, ohne zu wissen, wie der JDBC-Programmcode auszusehen hat.

10.1.1.1. Membervariablen

```
1 private static DBManager instance = null;  
2 private static final BasicPasswordEncryptor PASSWORD_ENCRYPTOR = new  
    BasicPasswordEncryptor();  
3 private static Connection con = null;
```

- **instance:** wird für das Singleton-Pattern zur Erstellung des DBManager-Objekts verwendet. Speichert das DBManager Objekt und stellt über das Singleton-Pattern fest, dass keine zweite Instanz dieses Objekts erstellt werden kann.
- **PASSWORD.ENCRYPTOR:** eine Instanz der verwendeten Library BasicPasswordEncryptor. Wie der Name es schon hergibt wird dieses Objekt zum Verschlüsseln von Passwörtern verwendet. Hierbei kann mit der Methode encryptPassword(String password) das mitgegebene Passwort verschlüsselt und mit der Methode checkPassword(String plainPassord, String encryptedPassword) ein eingegebenes Passwort auf Richtigkeit überprüft werden.
- **con:** Die Verbindung mit der Datenbank über JDBC wird mittels Connection-Objekt gespeichert. Da dieses Objekt nur datenbankrelevanten Nutzen hat wird dieses somit gleich als Membervariable gespeichert.

10.1.1.2. Konstruktor

```
1 private DBManager() {  
2     con = getConnection();  
3 }
```

Das einzige was der Konstruktor in dieser Klasse macht ist die Membervariable con, welche vorher null ist, zu überschreiben. Dies wird über die Methode getConnection()

10. Implementierte Klassen

geregelt. Auffällig ist, dass dieser Konstruktor `private` deklariert ist und man somit nicht einfach über ihn eine neue Instanz in einer anderen Klasse erzeugen kann. Der Grund dafür ist, dass das Objekt nur über das Singleton-Pattern erzeugt werden soll.

10.1.1.3. getInstance

```
1 public static DBManager getInstance() {
2     if (instance == null) {
3         instance = new DBManager();
4     }
5     return instance;
6 }
```

Die `getInstance()`-Methode ist die eigentliche Methode zur Erzeugung der `DBManager`-Instanz. Hierbei handelt es sich um ein Singleton-Pattern. Sollte das `instance`-Objekt `null` sein wird der Konstruktor aufgerufen und das `instance`-Objekt überschrieben, falls nicht wird das `instance`-Objekt retourniert. Somit wird gewährleistet, dass nur eine Instanz des `DBManagers` erstellt werden kann.

10.1.1.4. getConnection

```
1 private Connection getConnection() {
2
3     //den Dateipfad fuer smartmirror.db in einem String abspeichern
4     String db = "jdbc:sqlite:sqlite/smartmirror.db";
5
6     try {
7         //anhand des Strings die Connection herstellen:
8         //wenn die Datei bereits existiert, wird lediglich die Connection
9         //hergestellt
10        //ansonsten wird die Datei erstellt und dann eine Connection hergestellt (
11        //leere Datei)
12        con = DriverManager.getConnection(db);
13
14        //abpruefen ob die Datei leer ist (neu), wenn ja, dann die Konfigurations-
15        //Methode
16        //zum auffuellen der Datenbank ablaufen lassen
17        if (isEmpty()) {
18            createDatabase();
19        }
20
21    } catch (SQLException e) {
22        e.printStackTrace();
23    }
24
25    return con;
26 }
```

Mit dieser Methode wird über das `Connection`-Objekt eine Datenbankverbindung zur Datenbank `smartmirror.db` erstellt. In Zeile 10 wird anhand des Dateipfads die Verbindung zur Datenbank hergestellt. Existiert diese Datei noch nicht, wird sie an dieser Stelle erstellt. In Zeile 14 wird überprüft, ob die Datenbank an dieser Stelle

10. Implementierte Klassen

erstellt wurde oder schon vorhanden ist. Wenn sie leer ist, wird die Konfigurationsmethode für die Datenbank aufgerufen. Anschließend wird die erstellte Datenbankverbindung retourniert.

10.1.1.5. releaseConnection

```
1 public void releaseConnection () {
2     try {
3         if (con != null) {
4             con.close();
5         }
6
7     } catch (SQLException e) {
8         e.printStackTrace();
9     }
10 }
```

Über diese Methode wird die Datenbankverbindung einfach über `con.close()` geschlossen, falls diese noch aufrecht steht.

10.1.1.6. isEmpty

```
1 private boolean isEmpty() throws SQLException {
2     //anfaenglich wird angenommen, dass die datenbank nicht leer ist
3     boolean result = false;
4     PreparedStatement stm = null;
5     ResultSet rs = null;
6     try {
7         //Zaehlt die Anzahl der User in der Datenbank
8         //Sollte die Tabelle nicht existieren, dann erhaelt man eine SQLException
9         String sql = "SELECT count(*) from user;";
10        stm = con.prepareStatement(sql);
11
12        rs = stm.executeQuery();
13
14        //sollte die user-Tablle existieren, aber noch kein User eingetragen sein (
15        //count = 0)
16        //wird die Konfiguration trotzdem durchgefuehrt
17        if (rs.next()) {
18            result = rs.getInt(1) == 0;
19        }
20
21        //catch wird ausgefuehrt, wenn die Datenbank leer ist
22    } catch (SQLException e) {
23        result = true;
24    } finally {
25        if (stm != null) stm.close();
26        if (rs != null) rs.close();
27    }
28    return result;
29 }
```

10. Implementierte Klassen

In dieser Methode wird überprüft ob die Datenbank neu erzeugt wurde oder bereits existiert. Hierbei wird die Anzahl der bestehenden Nutzer in der Datenbank ermittelt und verglichen, ob diese 0 entspricht. Da am Ende der Konfigurationsmethode `createDatabase()` zwanghaft ein Nutzer erstellt werden muss, gilt dies als Entscheidungskriterium.

Das Ergebnis dieser Methode ist die ein bool'scher Wert, welcher aussagt, ob die Datenbank leer ist (true) oder nicht (false).

10.1.1.7. createDatabase

```
1 private void createDatabase() throws SQLException {
2     System.out.println("Generating Database...");
3
4     //alle Tabellen loeschen, wenn diese existieren
5     String sql = "drop table if exists location;";
6     createQuery(sql);
7     sql = "drop table if exists user;";
8     createQuery(sql);
9     sql = "drop table if exists locationData;";
10    createQuery(sql);
11    sql = "drop table if exists alarm;";
12    createQuery(sql);
13
14    //die Tabellen user, location, locationData und alarm erstellen
15    sql = "create table location (\n"
16    + "locationID int primary key,\n"
17    + "location varchar(100) not null,\n"
18    + "urlString varchar(255) not null\n"
19    + ");";
20    createQuery(sql);
21
22    sql = "create table user (\n"
23    + "_id integer primary key,\n"
24    + "faceID varchar(255) not null unique,\n"
25    + "username varchar(50) not null unique,\n"
26    + "password varchar(50) not null,\n"
27    + "profilePicture varchar(255),\n"
28    + "locationID int not null,\n"
29    + "foreign key(locationID) references location(locationID)\n"
30    + ");";
31    createQuery(sql);
32
33    sql = "create table locationData (\n"
34    + "currentDateTime timestamp default current_timestamp,\n"
35    + "locationID int,\n"
36    + "description varchar(100) not null,\n"
37    + "icon varchar(255) not null,\n"
38    + "temperature double not null,\n"
39    + "humidity double not null,\n"
40    + "windSpeed double not null,\n"
41    + "windAngle double not null,\n"
42    + "timezone int not null,\n"
43    + "primary key(currentDateTime, locationID),\n"
44    + "foreign key(locationID) references location(locationID)\n"
45    + ");";
```

10. Implementierte Klassen

```
46     createQuery(sql);
47
48     sql = "create table alarm (\n"
49     + "_id integer,\n"
50     + "userID int,\n"
51     + "alarmDateTime timestamp default current_timestamp,\n"
52     + "audioFile varchar(255) not null,\n"
53     + "alarmText varchar(255) not null,\n"
54     + "primary key(_id, userID),\n"
55     + "foreign key(userID) references user(_id) "
56     + ");";
57     createQuery(sql);
58
59     //citylist.json einlesen und in location eintragen
60     readCityList();
61
62     //user anlegen
63     System.out.println("Please create the first user: ");
64     createUser();
65     System.out.println("User successfully created!");
66 }
```

In dieser Methode werden zunächst alle Tabellen aus dem Datenbankmodell gelöscht, sollten diese existieren. Daraufhin werden sie wieder laut Datenbankschema erstellt. Anschließend wird über die Methode `readCityList()` in Zeile 60 die location-Tabelle befüllt und über die Methode `createUser()` in Zeile 64 der erste Nutzer erstellt.

10.1.1.8. createQuery

```
1 private void createQuery(String sql) throws SQLException {
2     Statement stmt = con.createStatement();
3     stmt.execute(sql);
4 }
```

Diese Methode dient zum Erstellen von bestimmten Datenbankstrukturen, wie z.B.: Tabellen. Der mitgegebene SQL-Code wird hier einfach mit `stmt.execute(sql)` in Zeile 3 ausgeführt.

10.1.1.9. readCityList

```
1 private void readCityList() throws SQLException {
2     try {
3         Gson gson = new Gson();
4
5         //Reader erstellen, welcher die json-datei abliest
6         Reader reader = Files.newBufferedReader(Paths.get("citylist/city.list.json"
7             ));
8
9         //json Datei in eine ArrayList speichern
10        ArrayList<?> cities = gson.fromJson(reader, ArrayList.class);
11
12        //apiKey lokal abspeichern (fuer die Bildung von urlString wichtig)
13        String apiKey = "*****";
```

10. Implementierte Klassen

```
13
14 //jeden Ort durchgehen und die Werte in die Datenbank speichern
15 for (int i = 0; i < cities.size(); i++) {
16     //parameter fuer die Darstellung des Fortschritts
17     int currentLocation = i+1;
18     DecimalFormat f = new DecimalFormat("0.00");
19     double percent = (currentLocation/(double)cities.size()) * 100;
20
21     System.out.println("Reading city " + (i+1) + "/" + cities.size() + " ("
22         + f.format(percent) + "%)...");
23     //aus dem i-ten Eintrag des Arrays eine eingene Map bilden
24     //also fuer jeden einzelnen Ort
25     LinkedHashMap<String, Object> map = (LinkedHashMap<String, Object>)
26         cities.get(i);
27
28     //aus dieser Map lassen sich nun folgende Werte auslesen
29     int id = (int)Double.parseDouble(map.get("id").toString());
30     String name = map.get("name").toString();
31     String urlString = "http://api.openweathermap.org/data/2.5/weather?id="
32         + id +
33         "&appid=" + apiKey + "&units=metric";
34
35     Location l = new Location(id, name, urlString);
36
37     //diese Werte nun in die Datenbank abspeichern
38     insertIntoLocation(l);
39 }
40
41 System.out.println("Finished reading all cities!\n");
42 } catch (IOException e) {
43     e.printStackTrace();
44 }
45 }
```

Diese Methode dient zur Abspeicherung aller Städte, mit denen ein API-Call möglich ist, in die Datenbank. Grundlage hierfür ist die Datei city.list.json. In dieser JSON-Datei sind die Grundwerte aller möglichen Städte eingetragen.

Zunächst wird die gesamte JSON Datei mit einem Reader abgelesen (Zeile 6) und die Städte in einer ArrayList abgespeichert (Zeile 9). Als nächstes wird für jeden Eintrag dieser ArrayList die CityID, der Name des Ortes und die URL des API-Calls herausgefiltert und dadurch ein Location-Objekt erstellt (Zeile 32). Anhand dieses Objekts wird ein Eintrag in der Datenbank erstellt (Zeile 37).

Da diese JSON-Datei insgesamt 209.579 Städte umfasst, dauert dieser Prozess insgesamt 25min und 45sec.

10.1.1.10. insertIntoLocation

```
1 public void insertIntoLocation(Location l) throws SQLException {
2     String sql = "insert into location (" +
3         "locationID, location, urlString) " +
4         "values (?, ?, ?);";
5     PreparedStatement stm = null;
6     try {
```

10. Implementierte Klassen

```
7      stm = con.prepareStatement(sql);
8      stm.setInt(1, l.getLocationID());
9      stm.setString(2, l.getLocation());
10     stm.setString(3, l.getUrlString());
11     stm.executeUpdate();
12
13     } finally {
14         if (stm != null) {
15             stm.close();
16         }
17     }
18 }
```

Über diese Methode wird ein Datenbankeintrag für die Tabelle Location erstellt. Als Parameter erhält die Methode ein Location-Objekt. Die Werte zum Eintragen in die Datenbank werden über getter-Methoden ermittelt und in das PreparedStatement eingefügt. Mit dem Befehl `stm.executeUpdate()` in Zeile 11 wird der Eintrag letztendlich erstellt.

10.1.1.11. createUser

```
1 public void createUser() throws SQLException {
2     boolean inputOk;
3
4     //Username einlesen
5     String username;
6     do {
7         Scanner sc = new Scanner(System.in);
8         System.out.print("Username: ");
9         username = sc.next();
10
11         inputOk = true;
12         if (usernameExists(username)) {
13             System.out.println("The username '" + username + "' already exists!
14             Please choose another one!");
15             inputOk = false;
16         }
17     } while (!inputOk);
18
19     //Passwort speichern
20     String password;
21     String confirmPassword;
22
23     do {
24         Scanner sc = new Scanner(System.in);
25         System.out.print("Password: ");
26         password = sc.next();
27         password = PASSWORD_ENCRYPTOR.encryptPassword(password);
28
29         System.out.print("Confirm Password: ");
30         confirmPassword = sc.next();
31
32         inputOk = true;
33         if (!(PASSWORD_ENCRYPTOR.checkPassword(confirmPassword, password))) {
34             System.out.println("Please enter the same password twice!");
35             inputOk = false;
36         }
37     } while (!inputOk);
38 }
```


10. Implementierte Klassen

```
35     }
36 } while (!inputOk);
37
38 //TODO: Gesicht abfragen
39 //als faceID wird die Datei des dazugehoerigen Gesichts abgespeichert
40 //hier als demo-code nur username.txt
41 String faceID = "faceids/" + username + ".txt";
42
43 //Location auswaehlen
44 String location;
45 int locationID;
46 do {
47     Scanner sc = new Scanner(System.in);
48     System.out.print("Select Location for Weather Data: ");
49     location = sc.next();
50
51     Location l = getLocationByName(location);
52     locationID = l.getLocationID();
53     inputOk = true;
54     if (locationID == -1) {
55         System.out.println("The location: '" + location + "' doesn't exist!");
56         inputOk = false;
57     }
58 }
59
60 } while (!inputOk);
61
62 //user in die Datenbank speichern
63 User u = new User(-1, faceID, username, password, "", locationID);
64 insertIntoUser(u);
65 }
```

Diese Methode wird zum Erstellen eines Nutzers verwendet. Hierbei sollen folgende Werte mit bestimmten Bedingungen abgefragt werden:

- **Username:** Dieser darf noch nicht in der Datenbank vorhanden sein, da dieser eindeutig sein muss.
- **Password:** Dieses muss zweimal hintereinander korrekt eingegeben werden. Bei der Eingabe wird dieses direkt über den PASSWORD_ENCRPYTOR verschlüsselt.
- **FaceID:** Hierbei wird auf die Datei mit den Gesichtswerten hingewiesen (muss noch implementiert werden).
- **Location:** Der eingegebene Standort muss in der Datenbank vorhanden sein. Ist dies der Fall so wird die locationID beim Nutzer mitgespeichert.

Wenn all diese Eingaben korrekt sind wird in Zeile 63 das User-Objekt erstellt und in der nächsten Zeile ein Eintrag in der Datenbank erstellt.

10.1.1.12. usernameExists

```
1 public boolean usernameExists(String username) throws SQLException {
2     boolean result = false;
3     PreparedStatement stm = null;
4     ResultSet rs = null;
```

10. Implementierte Klassen

```
5      try {
6          String sql = "select count(*) from user " +
7              "where username = ?;";
8          stm = con.prepareStatement(sql);
9          stm.setString(1, username);
10         rs = stm.executeQuery();
11         if (rs.next()) {
12             result = rs.getInt(1) == 1;
13         }
14     } finally {
15         if (stm != null) stm.close();
16         if (rs != null) rs.close();
17     }
18     return result;
19 }
```

In dieser Methode wird überprüft, ob ein Datenbankeintrag für den mitgegebenen Usernamen existiert. Falls dies der Fall ist, wird der bool'sche Wert **true** retourniert, ansonsten **false**.

10.1.1.13. getLocationByName

```
1 public Location getLocationByName(String name) throws SQLException {
2     PreparedStatement stm = null;
3     ResultSet rs = null;
4     Location l = new Location();
5     try {
6         String sql = "select locationID, location, urlString " +
7             "from location where location = ?;";
8         stm = con.prepareStatement(sql);
9         stm.setString(1, name);
10        rs = stm.executeQuery();
11        if (rs.next()) {
12            l.setLocationID(rs.getInt(1));
13            l.setLocation(rs.getString(2));
14            l.setUrlString(rs.getString(3));
15        }
16    }
17    finally {
18        if (stm != null) stm.close();
19        if (rs != null) rs.close();
20    }
21    return l;
22 }
```

Mithilfe dieser Methode kann ein Location-Objekt aufgrund des Namens des Ortes erzeugt werden. Dieses Objekt beinhaltet vor allem die locationID, welche für weitere Programmabläufe von Bedeutung sein kann.

10.1.1.14. insertIntoUser

```
1 public void insertIntoUser(User u) throws SQLException {
2     //userID = auto_increment -> Default-Wert
```

10. Implementierte Klassen

```
3 String sql = "insert into user (" +
4 "faceID, username, password, profilePicture, locationID) " +
5 "values (?, ?, ?, ?, ?)";
6 PreparedStatement stm = null;
7 try {
8     stm = con.prepareStatement(sql);
9     stm.setString(1, u.getFaceID());
10    stm.setString(2, u.getUsername());
11    stm.setString(3, u.getPassword());
12    stm.setString(4, u.getProfilePicture());
13    stm.setInt(5, u.getLocationID());
14    stm.executeUpdate();
15 }
16 finally {
17     if (stm != null) stm.close();
18 }
19 }
```

Diese Methode erzeugt einen neuen Eintrag in der User-Tabelle mithilfe des mitgegebenen User-Objekts. Das PreparedStatement wird hierbei mit getter-Methoden befüllt und in Zeile 14 ausgeführt. Hierbei muss die UserID nicht mit angegeben werden, da diese auto_increment definiert ist.

10.1.1.15. userLoginIsValid

```
1 public boolean userLoginIsValid(String username, String password) throws
   SQLException {
2     boolean result = false;
3     PreparedStatement stm = null;
4     ResultSet rs = null;
5     try {
6         String sql = "select password from user " +
7             "where username = ?";
8         stm = con.prepareStatement(sql);
9         stm.setString(1, username);
10        rs = stm.executeQuery();
11        if (rs.next()) {
12            result = PASSWORD_ENCRYPTOR.checkPassword(password, rs.getString(1));
13        }
14    }
15    finally {
16        if (stm != null) stm.close();
17        if (rs != null) rs.close();
18    }
19    return result;
20 }
```

Mit der Methode userLoginIsValid kann überprüft werden, ob der eingegebene Username mit dem eingegebenen Passwort zusammenpasst. Hierbei wird das Passwort des Nutzers anhand seines Usernamens aus der Datenbank ermittelt. Dieses wird dann mit dem eingegebenen Passwort verglichen. Als Ergebnis wird ein bool'scher Wert retourniert.

10. Implementierte Klassen

10.1.1.16. getLocationByID

```
1 public Location getLocationByID(int id) throws SQLException {
2     PreparedStatement stm = null;
3     ResultSet rs = null;
4     Location l = new Location();
5     try {
6         String sql = "select locationID, location, urlString " +
7             "from location where locationID = ?;";
8         stm = con.prepareStatement(sql);
9         stm.setInt(1, id);
10        rs = stm.executeQuery();
11        if (rs.next()) {
12            l.setLocationID(rs.getInt(1));
13            l.setLocation(rs.getString(2));
14            l.setUrlString(rs.getString(3));
15        }
16    }
17    finally {
18        if (stm != null) stm.close();
19        if (rs != null) rs.close();
20    }
21    return l;
22 }
```

Anhand der locationID kann mithilfe dieser Methode das dazugehörige Location-Objekt retourniert werden. Es werden alle Werte der Location aus der Datenbank geholt und ein Location-Objekt gebildet. Dieses wird daraufhin retourniert.

10.1.1.17. insertIntoLocationData

```
1 public void insertIntoLocationData(LocationData ld) throws SQLException {
2     //currentTime -> Default-Wert
3     String sql = "insert into locationData (" +
4         "locationID, description, icon, temperature, humidity, windSpeed, windAngle, " +
5         "timeZone) " +
6         "values (?, ?, ?, ?, ?, ?, ?, ?);";
7     PreparedStatement stm = null;
8     try {
9         stm = con.prepareStatement(sql);
10        stm.setInt(1, ld.getLocationID());
11        stm.setString(2, ld.getDescription());
12        stm.setString(3, ld.getIcon());
13        stm.setDouble(4, ld.getTemperature());
14        stm.setDouble(5, ld.getHumidity());
15        stm.setDouble(6, ld.getWindSpeed());
16        stm.setDouble(7, ld.getWindAngle());
17        stm.setInt(8, ld.getTimeZone());
18        stm.executeUpdate();
19    }
20    finally {
21        if (stm != null) stm.close();
22    }
23 }
```

10. Implementierte Klassen

Diese Methode erstellt einen Datenbankeintrag für die Tabelle LocationData. Als Grundlage für den Eintrag dient das mitgegebene LocationData-Objekt. Da der Wert `currentDateTime` standardmäßig beim Eintrag gesetzt wird, muss dieser nicht mit angegeben werden.

10.1.1.18. getLatestLocationData

```
1 public LocationData getLatestLocationData(int locationID) throws SQLException {
2     PreparedStatement stm = null;
3     ResultSet rs = null;
4     LocationData ld = new LocationData();
5     try {
6         String sql = "select * " +
7             "from locationData where locationID = ? order by currentDateTime desc limit
8                 1;";
9         stm = con.prepareStatement(sql);
10        stm.setInt(1, locationID);
11        rs = stm.executeQuery();
12        if (rs.next()) {
13            ld.setDateTime(rs.getTimestamp(1));
14            ld.setLocationID(rs.getInt(2));
15            ld.setDescription(rs.getString(3));
16            ld.setIcon(rs.getString(4));
17            ld.setTemperature(rs.getDouble(5));
18            ld.setHumidity(rs.getDouble(6));
19            ld.setWindSpeed(rs.getDouble(7));
20            ld.setWindAngle(rs.getDouble(8));
21            ld.setTimeZone(rs.getInt(9));
22        }
23    } finally {
24        if (stm != null) stm.close();
25        if (rs != null) rs.close();
26    }
27    return ld;
28 }
```

Um Aufschluss über die aktuellen Wetterdaten eines Standorts zu bekommen, benötigen wir eine Methode, welche den neuesten Eintrag der LocationData Tabelle retourniert. Dieser Aufruf geschieht über die `locationID`. Hierbei werden alle Einträge eines Standorts über den Eintragungszeitpunkt absteigend sortiert und nur der erste Eintrag ausgegeben. Somit wird sichergestellt, dass es sich hierbei um den neusten Eintrag handelt. Anschließend wird über diesen Eintrag ein LocationData-Objekt erstellt und dieses retourniert.

10.1.1.19. changeUserLocation

```
1 private void changeUserLocation(User u, int newLocationID) throws SQLException {
2     String sql = "update user " +
3         "set locationID = ? " +
4         "where _id = ?;";
5     PreparedStatement stm = null;
6     try {
```

10. Implementierte Klassen

```
7      u.setLocationID(newLocationID);
8      stm = con.prepareStatement(sql);
9      stm.setInt(1, u.getLocationID());
10     stm.setInt(2, u.getUserID());
11     stm.executeUpdate();
12 }
13 finally {
14     if (stm != null) stm.close();
15 }
16 }
```

In dieser Methode wird der Wunschstandort eines mitgegebenen Nutzers über die locationID geändert. Dies wird über einen update-Befehl in der Datenbank geregelt und die locationID geändert.

10.1.1.20. getUserByUsername

```
1 public User getUserByUsername(String username) throws SQLException {
2     PreparedStatement stm = null;
3     ResultSet rs = null;
4     User u = new User();
5     try {
6         String sql = "select * " +
7             "from user where username = ?";
8         stm = con.prepareStatement(sql);
9         stm.setString(1, username);
10        rs = stm.executeQuery();
11        if (rs.next()) {
12            u.setUserID(rs.getInt(1));
13            u.setFaceID(rs.getString(2));
14            u.setUsername(rs.getString(3));
15            u.setPassword(rs.getString(4));
16            u.setProfilePicture(rs.getString(5));
17            u.setLocationID(rs.getInt(6));
18        }
19    }
20    finally {
21        if (stm != null) stm.close();
22        if (rs != null) rs.close();
23    }
24    return u;
25 }
```

Durch diese Methode kann ein User einfach über seinen Usernamen aus der Datenbank ermittelt werden. Da dieser Wert eindeutig ist, wird maximal nur ein Nutzer retourniert. Anschließend wird ein User-Objekt erstellt und dieses retourniert.

10.1.1.21. getUserByID

```
1 public User getUserByID(int id) throws SQLException {
2     PreparedStatement stm = null;
3     ResultSet rs = null;
4     User u = new User();
```

10. Implementierte Klassen

```
5      try {
6          String sql = "select * " +
7              "from user where _id = ?;";
8          stm = con.prepareStatement(sql);
9          stm.setInt(1, id);
10         rs = stm.executeQuery();
11         if (rs.next()) {
12             u.setUserID(rs.getInt(1));
13             u.setFaceID(rs.getString(2));
14             u.setUsername(rs.getString(3));
15             u.setPassword(rs.getString(4));
16             u.setProfilePicture(rs.getString(5));
17             u.setLocationID(rs.getInt(6));
18         }
19     }
20     finally {
21         if (stm != null) stm.close();
22         if (rs != null) rs.close();
23     }
24     return u;
25 }
```

Im Grunde funktioniert diese Methode genau gleich wie `getUserByUsername`. Jedoch wird hierbei der Nutzer nicht anhand seines Usernamens ermittelt, sondern anhand seiner UserID.

10.1.1.22. insertIntoAlarm

```
1 public void insertIntoAlarm(Alarm a) throws SQLException {
2     //alarmid = auto_increment -> Default-Wert
3     String sql = "insert into alarm (" +
4         "userID, alarmDateTime, audioFile, alarmText) " +
5         "values (?, ?, ?, ?);";
6     PreparedStatement stm = null;
7     try {
8         stm = con.prepareStatement(sql);
9         stm.setInt(1, a.getUserID());
10        stm.setTimestamp(2, a.getDateTime());
11        stm.setString(3, a.getAudioFile());
12        stm.setString(4, a.getAlarmText());
13        stm.executeUpdate();
14    }
15    finally {
16        if (stm != null) stm.close();
17    }
18 }
```

Bei dieser Methode handelt es sich um die insert-Methode für die Alarm-Tabelle. Als Parameter wird ein Alarm-Objekt mitgegeben. Die alarmID ist hierbei `auto_increment`, weshalb diese nicht mit angegeben werden muss.

10.1.1.23. getAlarmByID

10. Implementierte Klassen

```
1 public Alarm getAlarmByID(int id) throws SQLException {
2     PreparedStatement stm = null;
3     ResultSet rs = null;
4     Alarm a = new Alarm();
5     try {
6         String sql = "select * " +
7             "from alarm where alarmID = ?;";
8         stm = con.prepareStatement(sql);
9         stm.setInt(1, id);
10        rs = stm.executeQuery();
11        if (rs.next()) {
12            a.setAlarmID(rs.getInt(1));
13            a.setUserID(rs.getInt(2));
14            a.setDateTime(rs.getTimestamp(3));
15            a.setAudioFile(rs.getString(4));
16            a.setAlarmText(rs.getString(5));
17        }
18    }
19    finally {
20        if (stm != null) stm.close();
21        if (rs != null) rs.close();
22    }
23    return a;
24 }
```

Hiermit kann ein Alarm-Objekt aufgrund der alarmID erzeugt werden. Hierbei wird wiederum ein SQL-Select ausgeführt und ein Alarm-Objekt retourniert.

10.1.1.24. getAllAlarmsByUser

```
1 public ArrayList<Alarm> getAllAlarmsByUser(User u) throws SQLException {
2     PreparedStatement stm = null;
3     ResultSet rs = null;
4     ArrayList<Alarm> result = new ArrayList<Alarm>();
5     try {
6         String sql = "select * from alarm where userID = ?;";
7         stm = con.prepareStatement(sql);
8         stm.setInt(1, u.getUserID());
9         rs = stm.executeQuery();
10        while(rs.next()) {
11            Alarm a = new Alarm();
12            a.setAlarmID(rs.getInt(1));
13            a.setUserID(rs.getInt(2));
14            a.setDateTime(rs.getTimestamp(3));
15            a.setAudioFile(rs.getString(4));
16            a.setAlarmText(rs.getString(5));
17            result.add(a);
18        }
19    }
20    finally {
21        if (stm != null) stm.close();
22        if (rs != null) rs.close();
23    }
24    return result;
25 }
```


10. Implementierte Klassen

Da ein Nutzer laut Datenbankschema mehrere Alarmer setzen kann, ist eine Methode notwendig, welche alle Alarmer eines Nutzers retourniert. Hierbei wird ein SQL-Select ausgeführt und aus allen Datensätzen jeweils ein Alarm Objekt gebildet. All diese Objekte werden in einer ArrayList gespeichert und diese letztendlich retourniert.

10.1.1.25. getUserByFaceID

```
1 public User getUserByFaceID(String faceID) throws SQLException {
2     PreparedStatement stm = null;
3     ResultSet rs = null;
4     User u = new User();
5     try {
6         String sql = "select * " +
7             "from user where faceID = ?;";
8         stm = con.prepareStatement(sql);
9         stm.setString(1, faceID);
10        rs = stm.executeQuery();
11        if (rs.next()) {
12            u.setUserID(rs.getInt(1));
13            u.setFaceID(rs.getString(2));
14            u.setUsername(rs.getString(3));
15            u.setPassword(rs.getString(4));
16            u.setProfilePicture(rs.getString(5));
17            u.setLocationID(rs.getInt(6));
18        }
19    }
20    finally {
21        if (stm != null) stm.close();
22        if (rs != null) rs.close();
23    }
24    return u;
25 }
```

Da die faceID in unserer Datenbank genauso wie der Username eindeutig ist, kann auch ein eindeutiger Datenbankcall anhand der faceID getätigt werden. Hierbei wird genau gleich vorgegangen, wie bei der Methode getUserByUsername, nur dass die mitgegebene faceID als Auswahlparameter dient.

10.1.1.26. changeUserProfilePicture

```
1 private void changeUserProfilePicture(User u, String newProfilePicture) throws
2     SQLException {
3     String sql = "update user " +
4         "set profilePicture = ? " +
5         "where userID = ?;";
6     PreparedStatement stm = null;
7     try {
8         u.setProfilePicture(newProfilePicture);
9         stm = con.prepareStatement(sql);
10        stm.setString(1, u.getProfilePicture());
11        stm.setInt(2, u.getUserID());
12        stm.executeUpdate();
13    }
14    finally {
```

10. Implementierte Klassen

```
14         if (stm != null) stm.close();
15     }
16 }
```

Mit dieser Methode kann der Datenbankeintrag eines Nutzers geändert werden. Dies bezieht sich in diesem Fall implizit auf das Profilbild. Der Dateipfad des neuen Profilbilds wird als Parameter mitgegeben und dieser folglich in der Datenbank eingetragen.

10.1.1.27. login

```
1 public User login() throws SQLException {
2     User u = new User();
3     System.out.println("Login");
4     boolean loginOk;
5
6     //Username einlesen
7     String username;
8     String password;
9     do {
10         Scanner sc = new Scanner(System.in);
11         System.out.print("Username: ");
12         username = sc.next();
13
14         System.out.print("Password: ");
15         password = sc.next();
16
17         loginOk = true;
18
19         if (!(userLoginIsValid(username, password))) {
20             System.out.println("Invalid Login! Try another username or password!");
21             loginOk = false;
22         }
23         else {
24             System.out.println("Login successful! Welcome " + username + "!");
25             u = getUserByUsername(username);
26         }
27     } while (!loginOk);
28     return u;
29 }
30 }
```

Mit dieser Methode kann sich ein Nutzer im System einloggen. Hierbei muss dieser seinen Usernamen und sein Passwort eingeben. Diese beiden Parameter werden dann über die Methode `userLoginIsValid` auf ihre Richtigkeit überprüft. Retourniert wird hierbei das dazugehörige User-Objekt, falls dieses existieren soll.

10.1.1.28. changeLocation

```
1 public void changeLocation(User u) throws SQLException {
2     boolean inputOk;
3     String newLocation;
4     int newLocationID;
```

10. Implementierte Klassen

```
5      do {
6          Scanner sc = new Scanner(System.in);
7          System.out.print("Select Location for Weather Data: ");
8          newLocation = sc.next();
9
10
11         Location l = getLocationByName(newLocation);
12         newLocationID = l.getLocationID();
13         inputOk = true;
14         if (newLocationID == -1) {
15             System.out.println("The location: '" + newLocation + "' doesn't exist!"
16                                 );
17             inputOk = false;
18         }
19     } while (!inputOk);
20
21     changeUserLocation(u, newLocationID);
22     System.out.println("Location for '" + u.getUsername() + "' changed to: '" +
23         newLocation + "'!");
24 }
```

Mit dieser Methode kann eine neue Location für einen den mitgegebenen Nutzer abgeändert werden. Hierbei geht es nicht dezidiert um die Änderung des Datenbankeintrags, sondern eher um die Ermittlung der benötigten Daten für diesen. Der Nutzer gibt hierbei über die Konsole seinen neuen Wunschstandort ein. Über die Methode `getLocationByName` in der Zeile 11 kann die `locationID` über den Standortnamen ermittelt werden. Danach wird überprüft, ob dieser Standort existiert. Falls nicht, muss der Standort erneut eingegeben werden. Anschließend wird die Methode `changeUserLocation` mit den ermittelten Werten ausgeführt.

10.1.1.29. changeProfilePicture

```
1 public void changeProfilePicture(User u) throws SQLException {
2     //TODO: Bild einlesen
3     String newProfilePicture = "profilepictures/test.jpg";
4
5     changeUserProfilePicture(u, newProfilePicture);
6 }
```

In dieser Methode soll ein neues Profilbild für die Abänderung des Datenbankeintrags ermittelt werden. Da die Frontendstruktur noch nicht vorhanden ist, muss diese Methode noch aufgrund dessen überschrieben werden. Anschließend wird für die tatsächliche Änderung des Datenbankeintrags die Methode `changeUserProfilePicture` aufgerufen.

10.1.2. class Location, User, LocationData, Alarm

Die vier Klassen **Location**, **User**, **LocationData** und **Alarm** dienen als Mapping-Klassen zum Datenbankschema. Die einzelnen Tabellen der Datenbank werden hier durch Membervariablen repräsentiert. Bei allen Methoden handelt es sich um automatisch generierte Methoden von Eclipse. Da die Struktur dieser vier Klassen im Grunde dieselbe ist werden alle vier Klassen durch die Klasse **Alarm** näher erklärt.

10.1.2.1. Membervariablen

```
1 private int alarmID;  
2 private int userID;  
3 private Timestamp dateTime;  
4 private String audioFile;  
5 private String alarmText;
```

Wie oben bereits beschrieben werden in den Membervariablen die einzelnen Tabellen der Datenbank repräsentiert. Hierbei sollte der Datentyp der einzelnen Variablen mit dem Datenbankschema übereinstimmen (für spezielle Datentypen, wie Timestamp, gibt es in dem java.sql-Package definierte Datentypen).

10.1.2.2. Konstruktor

Für alle vier Klassen wurden jeweils zwei Konstruktoren geschrieben. Einerseits ein Konstruktor, welcher jede Membervariable als Parameter erhält und diese mit den mitgegebenen Werten überschreibt:

```
1 public Alarm(int alarmID, int userID, Timestamp dateTime, String audioFile, String  
    alarmText) {  
2     this.alarmID = alarmID;  
3     this.userID = userID;  
4     this.dateTime = dateTime;  
5     this.audioFile = audioFile;  
6     this.alarmText = alarmText;  
7 }
```

Andererseits ein Konstruktor, welcher keine Parameter erhält. Hierbei werden unsinnige Standardwerte vergeben. Diese müssen im weiteren Lauf des Programms überschrieben werden:

```
1 public Alarm() {  
2     this.alarmID = -1;  
3     this.userID = -1;  
4     this.dateTime = null;  
5     this.audioFile = "";  
6     this.alarmText = "";  
7 }
```

10.1.2.3. getter-Methoden

10. Implementierte Klassen

```
1 public int getAlarmID() {  
2     return alarmID;  
3 }
```

Mittels getter-Methoden können private Membervariablen von anderen Klassen eingelesen werden. Hier wurde für jede Membervariable eine eigene getter-Methode geschrieben.

10.1.2.4. setter-Methoden

```
1 public void setAlarmID(int alarmID) {  
2     this.alarmID = alarmID;  
3 }
```

Setter-Methoden sind das Gegenstück zu getter-Methoden. Mit ihnen können private Membervariablen von anderen Klassen nicht eingelesen, sondern überschrieben werden. Auch hierbei wurde für jede Membervariable eine setter-Methode implementiert.

10.1.2.5. toString

```
1 public String toString() {  
2     return "Alarm [alarmID=" + alarmID + ", userID=" + userID + ", dateTime=" +  
3         dateTime + ", audioFile=" +  
4         + audioFile + ", alarmText=" + alarmText + "];"  
}
```

Durch die toString-Methode kann eine einfache Ausgabe der Objektparameter gemacht werden. Dies ist vor allem für Debug-Zwecke sehr sinnvoll.

10.1.2.6. equals

```
1 @Override  
2 public boolean equals(Object obj) {  
3     if (this == obj)  
4         return true;  
5     if (obj == null)  
6         return false;  
7     if (getClass() != obj.getClass())  
8         return false;  
9     Alarm other = (Alarm) obj;  
10    if (alarmID != other.alarmID)  
11        return false;  
12    if (alarmText == null) {  
13        if (other.alarmText != null)  
14            return false;  
15    } else if (!alarmText.equals(other.alarmText))  
16        return false;  
17    if (audioFile == null) {  
18        if (other.audioFile != null)  
19            return false;  
20    } else if (!audioFile.equals(other.audioFile))  
21        return false;  
22    return true;  
23 }
```

10. Implementierte Klassen

```
20     } else if (!audioFile.equals(other.audioFile))
21     return false;
22     if (dateTime == null) {
23         if (other.dateTime != null)
24             return false;
25     } else if (!dateTime.equals(other.dateTime))
26     return false;
27     if (userID != other.userID)
28     return false;
29     return true;
30 }
```

Mittels Aufruf der equals-Methode kann überprüft werden, ob es sich bei dem mitgegebenen Objekt um das gleiche Objekt handelt, mit der die Methode aufgerufen wurde. Im Grunde wird hierbei nur der Zeiger verglichen. Sollte dieser bei beiden Objekten gleich sein, muss es auch dasselbe Objekt sein.

10.1.2.7. hashCode

```
1  @Override
2  public int hashCode() {
3      final int prime = 31;
4      int result = 1;
5      result = prime * result + alarmID;
6      result = prime * result + ((alarmText == null) ? 0 : alarmText.hashCode());
7      result = prime * result + ((audioFile == null) ? 0 : audioFile.hashCode());
8      result = prime * result + ((dateTime == null) ? 0 : dateTime.hashCode());
9      result = prime * result + userID;
10     return result;
11 }
```

Mit der hashCode-Methode kann der sogenannte hashCode eines Objekts ermittelt werden.

Der Hash-Code ist ein integraler Wert, der verwendet wird, um Objekte in einem hash-basierten Container abzulegen oder sie in einem solchen Container zu finden. Die hash-basierten Container in Java sind `java.util.Hashtable`, `java.util.HashMap`, `java.util.HashSet` und deren Subklassen. [9].

10.2. package OpenWeatherMapAPI

10.2.1. class WeatherData

Im Allgemeinen ist die Klasse WeatherData für alle Operationen verantwortlich, welche die API-Requests der OpenWeatherMap-API betreffen. Sie ist vor allem verantwortlich für den Erhalt und die Aufbereitung der Daten aus dem API-Request.

Da für diese Klasse keine Objekte erzeugt werden, gibt es auch keine Membervariablen, keinen Konstruktoren und die Methoden werden durch das static Keyword objektunabhängig deklariert.

10.2.1.1. updateWeatherData

```

1 public static void updateWeatherData(DBManager dbm, int locationID) throws
   SQLException {
2     Location l = dbm.getLocationByID(locationID);
3     try {
4         //liest die gesamte Seite (JSON) und wandelt sie in einen String
5         String jsonString = readTextFromURL(l.getUrlString());
6
7         //Konvertieren des gesamten JSON-Strings als Map
8         Map<String, Object> respMap = jsonToMap(jsonString);
9
10        //Aufteilung in die 3 Maps: main, wind, weather
11        Map<String, Object> mainMap = jsonToMap(respMap.get("main").toString());
12        Map<String, Object> windMap = jsonToMap(respMap.get("wind").toString());
13        List<Map<String, Object>> weather = (List<Map<String, Object>>) (respMap.
            get("weather"));
14        Map<String, Object> weatherMap = weather.get(0);
15
16        //Parameter der Wetterdaten auslesen
17        String description = weatherMap.get("main").toString();
18        String icon = weatherMap.get("icon").toString();
19        icon = "icons/" + icon + ".png";
20        double temperature = Double.parseDouble(mainMap.get("temp").toString());
21        double humidity = Double.parseDouble(mainMap.get("humidity").toString());
22        double windSpeed = Double.parseDouble(windMap.get("speed").toString());
23        double windAngle = Double.parseDouble(windMap.get("deg").toString());
24        String tz = respMap.get("timezone").toString();
25        tz = tz.replace(".", "");
26        int timeZone = Integer.parseInt(tz);
27
28        //die Wetterdaten in die Datenbank eintragen
29        LocationData ld = new LocationData(null, locationID, description, icon,
            temperature, humidity, windSpeed, windAngle, timeZone);
30        dbm.insertIntoLocationData(ld);
31
32    } catch (IOException e) {
33        System.out.println(e.getMessage());
34    }
35 }

```

Diese Methode erstellt einen neuen Datenbankeintrag für die LocationData-Tabelle in

10. Implementierte Klassen

Abhängigkeit von der locationID. Die dafür benötigten Werte werden über eine API-Request mit der OpenWeatherMap-API ermittelt. Der API-Call erfordert die entsprechende URL, welche aus einem Location-Objekt ermittelt werden kann. Der API-Call liefert eine JSON-Antwort, aus welcher wir Informationen ableiten können.

Zunächst wird die erhaltene JSON-Antwort abgelesen und in einem String gespeichert. Danach wird aus diesem String eine Map generiert. Diese Map lässt sich dann in die drei weiteren Maps **main**, **wind** und **weather** aufteilen, aus welchen wir dann unsere benötigten Werte erhalten:

- description
- icon
- temperature
- humidity
- windSpeed
- windAngle
- timeZone

Aufgrund all dieser Werte kann ein neues LocationData-Objekt erstellt werden (Zeile 29) und aus diesem ein Eintrag in der Datenbank (Zeile 30).

10.2.1.2. updateDateTime

```
1 public static String updateDateTime(DBManager dbm, int locationID) throws
   SQLException {
2     LocationData ld = dbm.getLatestLocationData(locationID);
3     Date currentDateTime = new Date();
4     Calendar c = Calendar.getInstance();
5
6     //setzt das Calender-Objekt auf die momentane Zeit
7     //Vorraussetzung: Die Zeit des PCs muss auf UTC eingestellt sein
8     c.setTime(currentDateTime);
9
10    //fuegt die Zeitverschiebung von UTC hinzu und setzt das Date-Objekt
        entsprechend neu
11    c.add(Calendar.SECOND, (ld.getTimeZone()));
12    currentDateTime = c.getTime();
13
14    //Zeit formatieren und this.dateTime neu setzen
15    SimpleDateFormat dateTimeFormatter = new SimpleDateFormat("dd. LLLL yyyy - HH:
        mm:ss");
16    //SimpleDateFormat timeFormatter = new SimpleDateFormat("HH:mm:ss");
17    String dateTime = dateTimeFormatter.format(currentDateTime);
18
19    return dateTime;
20    //time = timeFormatter.format(currentDateTime);
21    //printData();
22 }
```


10. Implementierte Klassen

Da die lokale Zeit für den gewünschten Standort von der Zeitzone abhängt, und diese über die API ermittelt werden kann, wird eine Methode benötigt, welche diese Werte aktualisiert. Hierbei werden die aktuellen Wetterdaten für den gewünschten Standort mit der locationID bestimmt. Wichtig aus diesen Wetterdaten ist hier nur die timeZone. Über die Java-Klassen Date und Calendar lässt sich die momentane UTC-Zeit ermitteln. Zu dieser wird nun der Wert von timeZone (Zektverschiebung zu UTC in Sekunden) addiert. Die Variable currentDateTime erhält somit die aktuelle Uhrzeit vom gewünschten Standort (Zeile 12).

Diese Zeit wird nun durch einen dateTimeFormatter formatiert und retourniert.

10.2.1.3. readTextFromURL

```
1 private static String readTextFromURL(String urlString) throws IOException {
2     StringBuilder result = new StringBuilder();
3     URL url = new URL(urlString);
4     URLConnection conn = url.openConnection();
5     BufferedReader rd = new BufferedReader(new InputStreamReader(conn.
6         getInputStream()));
7     String line;
8     while ((line = rd.readLine()) != null) {
9         result.append(line);
10    }
11    rd.close();
12    return result.toString();
13 }
```

Diese Methode öffnet die mitgegebene URL, welche zu einer JSON-Antwort führt und wandelt diese in einen String um. Eine Verbindung zur URL wird mittels URLConnection aufgebaut. Durch einen BufferedReader wird folglich jede Zeile der JSON-Antwort eingelesen und an einen StringBuilder hinzu gehängt. Dieser wird folglich in einen String umgewandelt und retourniert.

10.2.1.4. jsonToMap

```
1 private static Map<String, Object> jsonToMap(String str) {
2     Map<String, Object> map = new Gson().fromJson(str, new TypeToken<HashMap<String
3         , Object>>() {
4     }.getType());
5     return map;
6 }
```

Durch diese Methode kann ein JSON-String zu einer HashMap umgewandelt werden. Hierfür wird die GSON-Library verwendet.

10.3. TimerTasks

Die Java-Klasse `TimerTask` dient dazu, zeitliche Abläufe zu regeln. Mit `TimerTasks` können bestimmte Programmabläufe in bestimmten Intervallen oder zu bestimmten Zeiten ausgeführt werden.

In unserem Fall mussten zwei ;Methoden in bestimmten Intervallen aufgerufen werden:

- **updateWeatherData:** Stündlich sollen die aktuellen Wetterdaten eines Standorts in die Datenbank gespeichert werden. Falls der Nutzer aus jeglichem Grund keine Internetverbindung aufbauen kann, wird ihm der letzte Eintrag der `LocationData`-Tabelle angezeigt. Deshalb ist es wichtig diese Daten auf dem neust möglichem Stand zu halten. Eine stündliche Aktualisierung ist daher ein guter Zeitintervall.
- **updateDateTime:** Da sich dieser Wert jede Sekunde ändert, muss diese Methode auch jede Sekunde aufgerufen werden und ihr Rückgabewert angezeigt werden.

Im Programmcode sieht dies folgendermaßen aus:

```

1  TimerTask updateWeather = new TimerTask() {
2
3      @Override
4      public void run() {
5          try {
6              WeatherData.updateWeatherData(dbm, u1.getLocationID());
7          } catch (SQLException e) {
8              e.printStackTrace();
9          }
10     }
11
12 };
13 TimerTask updateDateTime = new TimerTask() {
14
15     @Override
16     public void run() {
17         try {
18             String dateTime = WeatherData.updateDateTime(dbm, u1.getLocationID());
19             //TODO: dateTime anzeigen
20         } catch (SQLException e) {
21             e.printStackTrace();
22         }
23     }
24
25 };
26 Timer t = new Timer();
27 t.schedule(updateWeather, 1000*60*60);
28 t.schedule(updateDateTime, 1000);

```

Beim Erzeugen der `TimerTask`-Objekte muss die Methode `run` überschrieben werden. In diese Methode muss beschrieben werden, was ausgeführt werden soll (in unserem Fall die oben genannten Methoden). Für die Angabe des Intervalls muss ein `Timer` definiert werden (Zeile 26). Dieser sorgt dafür, dass der `TimerTask` `updateWeather` stündlich ($1.000 \times 60 \times 60 \text{ms} = 3.600.000 \text{ms} = 1 \text{h}$) und der `TimerTask` `updateDateTime` sekundlich ($1.000 \text{ms} = 1 \text{s}$) ausgeführt wird.

Teil V.

Schlusswort

11. Aufgetretene Probleme

11.1. Suche der Werkstatt

Durch den Umstieg auf Distance-Learning bekamen wir das Schulgebäude in diesem Schuljahr selten zu Gesicht. Darunter fällt auch die Werkstatt. Hierbei gäbe es ohnehin ein Problem: Nahezu alle Werkstätten in der HTL Anichstraße legen ihren Fokus auf die Metallbearbeitung und nicht auf die von uns benötigte Holzbearbeitung. Zudem wäre es aufwendig gewesen alle benötigten Bauteile und Rohstoffe von Hopfgarten bis nach Innsbruck zu transportieren.

Eine Lösung für diese Probleme kam von Herrn Ilija Filipovic, welcher netterweise bereit war uns seine ausgeprägte Holzwerkstatt zur Verfügung zu stellen. Da dieser ebenfalls in Hopfgarten lebt, war der Transport der Rohstoffe, benötigten Bauteile und letztendlich des Endprodukts kein Problem. In dieser Werkstatt war es, durch die umfangreiche Ausstattung, möglich den Spiegelrahmen mit Leichtigkeit zu fertigen.

11.2. Lieferung der benötigten Hardware

Aufgrund der weltweiten Pandemie, welche uns vor allem ab März des Jahres 2020 erreichte, mussten wir im Allgemeinen mit vielen Herausforderungen kämpfen. Eines dieser Probleme war die Komplikation bei der Bestellung der Hardware. Beispielsweise fanden wir auf Amazon eine halbreflektierende Acrylglasscheibe, welche perfekt für unsere Zwecke gewesen wäre. Jedoch stammt dieser Artikel aus Amerika und wurde seit der erschwerten Lage mit der Pandemie nicht nach Österreich versandt. Da es keine ähnlichen Produkte gab, mussten wir kreativ sein und eine völlig transparente Acrylglasscheibe zu einer halbreflektierenden umformen.

Insgesamt wurde das gesamte Projekt durch genau solche Probleme teurer. Zudem wurden auch Lieferzeiten verlängert, wodurch wir einiges an Zeit verloren haben.

11.3. Fehlende Motivation

Der letzte Punkt, bei dem uns die COVID-19 Pandemie den Projektfortschritt erschwerte, war die Motivation, welche nicht durchgehend für dieses Projekt aufrecht erhalten werden konnte. Auch die beiden oben genannten Punkte sorgten dafür, dass die Motivation zu einigen Zeitpunkten nicht wirklich vorhanden war.

11. Aufgetretene Probleme

Gearbeitet wurde hauptsächlich während den Ferien (Herbst-, Weihnachts- und Semesterferien), da sich der restliche Schulstress zu dieser Zeit in Grenzen hielt. Während der normalen Schulzeit war es wegen dem stressigen Distance-Learning nahezu unmöglich, an dieses Projekt zu denken.

Jedoch konnte im Endeffekt die notwendige Motivation aufgebracht und ein stabiles Projekt ermöglicht werden.

12. Zukünftige Schritte

Das Projektteam dieser Diplomarbeit besteht grundsätzlich aus drei Mitgliedern, welche jeweils ihre eigenen Aufgabengebiete haben. Jedoch entschieden sich zwei Mitglieder die Abgabe ihrer Teile bis zum Herbst zu verschieben. Bei diesen Teilen handelt es sich hauptsächlich um das Frontend der Software und die Gesichtserkennung bzw. Gestiksteuerung. Diese müssen zukünftig noch ergänzt werden und werden daher in diesem Kapitel nur oberflächlich erklärt.

12.1. Frontend

Anhand der erstellten Backend-Struktur soll in diesem Teil der Großteil der Software erstellt werden. Hierunter fällt eine Erstellung einer sogenannten GUI (Graphical User Interface), auf der die wesentlichen Daten angezeigt werden und welche eine einfache Navigation des Nutzers bietet. Der Nutzer soll über diese zumindest ins Internet kommen, zwischen Profilen wechseln, Alarme setzen und Einstellungen abrufen können.

12.2. Gesichtserkennung und Gestiksteuerung

Da ein Grundproblem bei diesem Projekt die Navigation in der Software war, benötigen wir eine einfache Steuerung ohne Maus und Tastatur. Genauer gesagt benötigen wir eine Möglichkeit der Gesichtserkennung, durch die ein Einloggen des Nutzers ermöglicht wird. Die Navigation durch die Software soll über Handbewegungen geschehen. Somit müssen wir eine Gestiksteuerung implementieren, welche Handbewegungen erkennt und anhand dieser das Programm steuert.

Literaturverzeichnis

- [1] ComputerWeekly.de. Structured query language (sql).
<https://www.computerweekly.com/de/definition/Structured-Query-Language-SQL>, 2021. Abgerufen am 10. März 2021.
- [2] Christoph Dalitz. Sqlite: Datenbank-engine im kleinformaT.
<https://www.linux-magazin.de/ausgaben/2004/09/datenspeicher-fuer-sparsame/>, 2004. Abgerufen am 10. März 2021.
- [3] Martin Kompf. Die ersten schritte zur java programmierung.
<https://www.kompf.de/java/firststeps.html>, 2019. Abgerufen am 10. März 2021.
- [4] OpenWeatherMapAPI. Openweathermap. <https://openweathermap.org/>, 2020. Abgerufen am 11. März 2021.
- [5] Marcel Peters. Wofür braucht man java? https://praxistipps.chip.de/wofuer-braucht-man-java-einfach-erkluert_43880, 2018. Abgerufen am 10. März 2021.
- [6] selbermachen.de. Arbeiten mit der oberfräse. <https://selbermachen.de/tools-tipps/werkzeug/arbeiten-mit-der-oberfraese>, 2020. Abgerufen am 6. März 2021.
- [7] selbermachen.de. Handkreissägen: Das müssen sie wissen.
<https://selbermachen.de/tools-tipps/werkzeug/handkreissaegen-das-muessen-sie-wissen>, 2020. Abgerufen am 6. März 2021.
- [8] software.fandom.com. Creo parametric.
https://software.fandom.com/de/wiki/Creo_Parametric, 2020. Abgerufen am 8. März 2021.
- [9] Klaus Kreft und Angelika Langer. Hash-code-berechnung.
<http://www.angelikalanger.com/Articles/EffectiveJava/03.HashCode/03.HashCode.html>, 2002. Abgerufen am 8. März 2021.
- [10] Lisa Waschbusch und Elisa Mundt. Was bedeutet industrie 4.0? definition, merkmale und anwendung. <https://www.industry-of-things.de/was-bedeutet-industrie-40-definition-merkmale-und-anwendung-a-828236/> 2019. Abgerufen am 12. März 2021.
- [11] Stefan Luber und Stephan Augsten. Was ist sql?
<https://www.dev-insider.de/was-ist-sql-a-586264/>, 2017. Abgerufen am 10. März 2021.

Literaturverzeichnis

- [12] Wikipedia. Eclipse (ide).
[https://de.wikipedia.org/wiki/Eclipse_\(IDE\)](https://de.wikipedia.org/wiki/Eclipse_(IDE)), 2021. Abgerufen am 10. März 2021.
- [13] Wikipedia. Japansäge.
<https://de.wikipedia.org/wiki/Japans%C3%A4ge>, 2021. Abgerufen am 11. März 2021.
- [14] Wikipedia. Java-technologie.
<https://de.wikipedia.org/wiki/Java-Technologie>, 2021. Abgerufen am 10. März 2021.
- [15] Wikipedia. Mitteldichte holzfaserplatte.
https://de.wikipedia.org/wiki/Mitteldichte_Holzfaserplatte, 2021. Abgerufen am 6. März 2021.

Abbildungsverzeichnis

2.1.	Raspberry Pi 4	18
2.2.	Einwegspiegeleffekt mit Acrylglas	19
2.3.	AOC 22B1H Monitor	20
2.4.	Innenseite eines Monitors	21
2.5.	Grundlegende Positionierung auf der Innenplatte des Monitors	21
2.6.	Sunflower Bag Raspberry Pi USB 2.0 Free Driver Camera	22
2.7.	Audiocore AC870 Stereo Speakers	23
2.8.	Logitech Wireless Keyboard	24
3.1.	Handkreissäge	26
3.2.	Oberfräse	27
3.3.	Aufbau einer Kappsäge	28
3.4.	Bandsäge	30
3.5.	Schnitzmesser	31
3.6.	Schonhammer	31
3.7.	Akkubohrer	32
3.8.	Schleifmaschine	33
3.9.	Japansäge	34
4.1.	Aufbau der Java-Technologie	35
5.1.	Creo Logo	37
5.2.	Eclipse Logo	38
5.3.	Unterschied Datenbanksystem vs. Desktop-Datenbank	38
7.1.	Modelle der bestellten Bauteile	46
7.2.	Modell der vorderen Rahmenplatte	47
7.3.	Modell des oberen Rahmentails	48
7.4.	Modell des unteren Rahmentails	49
7.5.	Modell des seitlichen Rahmentails	49
7.6.	Modell der Stabilisierungsplatte für den Monitor	50
7.7.	Modell für die Stütze des Monitors	51
7.8.	Modell der hinteren Rahmenplatte	52
7.9.	Baugruppe Schritt 1	53
7.10.	Baugruppe Schritt 2	53
7.11.	Baugruppe Schritt 3	54
7.12.	Baugruppe Schritt 4	54
7.13.	Baugruppe in allen Ansichten	55
7.14.	Zeichnungsableitung der vorderen Rahmenplatte	56
7.15.	Zeichnungsableitung des oberen Rahmentails	57

Abbildungsverzeichnis

7.16. Zeichnungsableitung des unteren Rahmenteils	57
7.17. Zeichnungsableitung des seitlichen Rahmenteils	58
7.18. Zeichnungsableitung der Stabilisierungsplatte für den Monitor	58
7.19. Zeichnungsableitung der Stütze des Monitors	59
7.20. Zeichnungsableitung der hinteren Rahmenplatte	59
8.1. Vollendung des Schnitts durch Japansäge	61
8.2. Markierung der Bohrung für die Webcam	61
8.3. Schnitt des Webcamlochs mit Oberfräse	62
8.4. Ausnehmung zum Einspannen der Lautsprecher	64
8.5. Bohrung für die Dübel-Verbindung	64
8.6. Vollendeter Schnitt bei der Stabilisierungsplatte	65
8.7. Ausnehmung bei der Stütze für den Monitor	66
8.8. Befestigung der Platine	67
8.9. Stütze für die Webcam	68
8.10. Dübel-Verbindung	69
8.11. Platten-Verbindung	70
8.12. Interner Aufbau	70
8.13. Befestigung des Raspberry Pi	71
8.14. Sprühen des Spiegelrahmens	72
8.15. Fertig besprühter Spiegelrahmen	72
9.1. Datenbankschema	74

Tabellenverzeichnis